

Book Manuscript Critique and Revision Proposals

Complete visible exchange covering Chapters 1-5, including critique, figure assessments, corrected passages, and implementation notes.

Source

<https://claude.ai/share/eb8f286b-1293-46e9-bafd-f561330fc155>

This is a copy of a chat between Claude and Torbjörn. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Prepared from the public shared snapshot on 4 July 2026. Attachments marked as hidden were not available in the shared page and are represented by notices in this transcript.

Attachment hidden in shared chat The shared page does not expose the uploaded source file or image.

Please provide me with critique and proposals for chapter 1 of my book manuscript. Concentrate on structure, content and readability. Coherence and theoretical adequacy are important. Ignore LaTeX issues.

Claude Message 2

Chapter 1 is in good shape overall: the arc from vision to concretization to motivation to roadmap is sound, the register is right for Springer, and §1.5 in particular is doing serious intellectual work. The problems are of three kinds: one significant structural redundancy, a handful of placement and forward-reference issues, and consistency debt (terminology, spelling, citations). I'll take them in order of importance, with splice-ready proposals where a rewrite is warranted.

Structure

1. The two-modes material is presented three times. The sequential/concurrent duality gets a full treatment in §1.3.2 (with code), again in §1.5.1 (four subsubsections), and a third time in §1.6.2 (items 1 and 3, which re-summarize §1.5.1 nearly verbatim: "unification serves as the data contract; backtracking maps onto paginated retrieval"). Each pass has a distinct job — demonstration, motivation, framing — but the summary sentences repeat, and the conceptual-foundation-vs-implementation-inversion point is made twice (end of §1.3.2 and end of §1.5.1). I'd resolve it as follows: keep §1.5.1 as the full development; compress the closing paragraph of §1.3.2 by cutting the Chapter 4 ordering rationale (which is meta-commentary that belongs in Chapter 4's own introduction, and duplicates the inversion remark in §1.5.1); and shrink §1.6.2's items 1 and 3 to two sentences each with cross-references. Proposed replacement for the §1.3.2 closer:

latex

```
Together, these two modes define the ``Web'' in ``Web Prolog''. The sequential mode -- stateless querying over HTTP, with backtracking mapped onto paginated retrieval -- is the simpler and often sufficient choice, and the one closest to how the traditional Web already works. The concurrent mode -- actor creation and message passing over WebSocket connections -- extends the model to applications that require persistent state and ongoing interaction. Both modes use the same language, the same term-based data model, and the same unification-based semantics; they differ only in their transport assumptions and in the kinds of interaction they support. Chapter~\ref{sec:the-prolog-web} develops each mode in detail.
```

And for §1.6.2:

latex

```
\item \textbf{Bring logic-driven querying to the Web.}
A node stores a knowledge base, exposes it through a stateless HTTP API, and answers queries by unification and search (Section~\ref{sec:why-web-prolog}). This sequential interaction model requires no concurrent infrastructure and is usable by any HTTP client in any programming language.
```

latex

```
\item \textbf{Leverage Erlang-style concurrency for persistent interaction.}
For applications that need persistent services, ongoing conversations, or coordinated multi-agent behaviour, Web Prolog extends the sequential foundation with actor-based concurrency and distribution over WebSocket connections -- an extension of, not a replacement for, the sequential model.
```

2. §1.2 interrupts a natural continuation. §1.1 ends by promising a deeper understanding; §1.3 opens by delivering exactly that ("A deeper understanding of what the Prolog Trinity is all about..."). The inventors' tribute, charming as it is, sits between them as a digression — especially since Harel's relevance is explicitly deferred. I'd move §1.2 to directly precede §1.7, where it forms a coherent historical-contextual block with "Thirty years of Prolog on the Web," opening with a bridge like: "Before surveying three decades of Prolog on the Web, let us pay tribute to five people whose inventions form the basis for the work presented in this book." The Kowalski epigraph in §1.1 survives the move without difficulty.

3. §1.4 states the diagnosis without the symptoms. The section says Prolog's state is "far from ideal" and "documented in a series of papers," but the actual documented weaknesses (limited portability, systems fragmentation, the post-desktop JavaScript threat) only surface in §1.5.4, where they are contrasted with your answers. A reader of §1.4 gets a headline without content. Add one compact paragraph enumerating FYPB's principal concerns at the point where the papers are introduced; §1.5.4 can then contrast rather than introduce.

4. §1.5.4 does double duty and should be split. Its first half (components insufficient alone; the intersection as the paradigmatic claim) answers the subsection's title question. From "Figure~1.3 does not reveal anything about the implementation..." onward, the topic shifts to implementation diversity, the community-driven project, and federation. That second half deserves its own heading — and you already have the right name for it: `\subsection{Federation over convergence}`. This also rescues the excellent scope-note paragraph from being buried at the end of an overlong subsection.

Content and theoretical adequacy

5. The "mismatch surfaces at the point of invocation" claim will draw fire. In §1.5.1, the contract argument says that when client and node have diverged, "the mismatch surfaces at the point of invocation rather than downstream in deserialisation code." True in *location*, but the signal is unification failure — which is indistinguishable, at the point of invocation, from a principled empty answer set. A referee will note that a silent failure can be *harder* to diagnose than a loud deserialisation error. Since you already forward-reference the fuller account, one concessive clause immunizes the claim: "...including a precise account of how the different kinds of divergence manifest -- and of the care needed to distinguish principled failure from mere absence of data."

6. The echo interaction quietly presupposes that the toplevel is an actor. The reader sees `self/1` return a pid and `receive/1` retrieve a message *in a later query* — which only works if the toplevel persists between queries with a mailbox of its own. This is one of your book's genuinely distinctive points (the operational specialness of the toplevel), and Chapter 1 is the place to plant it. After the interaction, add:

latex

```
\noindent Note, in passing, that this interaction works only because the
toplevel itself is an actor: it has a pid of its own and a mailbox in
which the echo can wait between queries. The operational specialness of
the toplevel is a theme we return to in
Chapter~\ref{sec:prolog-agents}.
```

7. The agent taxonomy is left implicit. §1.3.1 calls the actor "the simplest type of agent"; §1.5.2 says nodes "are themselves agents." The reader can assemble the taxonomy, but shouldn't have to. One sentence in §1.3.1 fixes it: "In the taxonomy adopted in this book, *agent* is the umbrella term: Prolog actors and Prolog nodes are both agents, and toplevel actors and statechart actors are specialised kinds of actor." Relatedly, the parenthetical hedge about "agent" in the opening is doing a lot of work for a parenthesis, particularly given how loaded the word has become. I'd promote it to a footnote that explicitly disclaims both strong agency in the BDI sense and the current LLM usage of "agentic" — except where Chapter 8 makes the connection deliberately.

8. Two smaller substantive points. First, "cannot be given a logical interpretation" (of `echo_actor`) is defensible but absolute; "cannot be given a declarative reading in the usual sense" costs nothing and forestalls the linear-logic pedant. Second, "Actors without a logic substrate are just Erlang" — the rhetorical triple is effective and I'd keep it, but be aware it reads as faintly dismissive of a language the book otherwise venerates; if that worries you, "are Erlang by another name" softens without deflating.

Terminology and consistency

9. "Partition" is used in two senses within the chapter. Figure 1.3's caption and §1.3's opening use it geographically ("a tiny partition of the Prolog Web" = a small region), while §1.5.1 and §1.5.4 use it architecturally (the sequential/concurrent partitions — a load-bearing technical term for the rest of the book). Change the geographic uses to "fragment" or "corner" and reserve "partition" for the architectural sense.

10. "Profile" also carries two senses: the language profile ("a carefully crafted profile of the Prolog language," opening paragraph) and the node profiles (conformance levels, §1.5.3). Both usages are legitimate standards-world idiom, but within one book they will collide. Where ambiguity threatens, write "language profile" and "node profile" explicitly. Related: §1.5.1's "Three of the four node profiles introduced later in the book" and §1.5.2's "the less capable profiles" both use the concept before it has been glossed. At first mention, add an appositive: "-- graded conformance levels that specify which capabilities a node exposes to its clients --".

11. Unanchored forward references. "the PTCP" (§1.5.2) appears as a bare acronym before any definition — expand it at first use. "Two out of the three APIs driving the Prolog Trinity ecosystem" (§1.7) references machinery not yet introduced — either add "(introduced in Chapter~\ref{sec:prolog-agents})" or rephrase as "two of the three web APIs that, as we shall see, drive the ecosystem."

12. Spelling: American stragglers against the British-throughout decision. In this chapter I find: *reconceptualization* (opening sentence), *behavior/behavioral* (intro ¶2, §1.5.2 ×3, §1.9 ×2 — while §1.3.2 and §1.5.3 correctly have *behaviour*), *authorized* (§1.3.1), *specialized* (§1.3.2, §1.5.2 — against *specialise* in §1.5.3–4), *realization* (§1.5 opening — against *realisation* ×2 in §1.5.4), *optimization* (§1.5.2), *decentralized* (§1.6, §1.9), *standardization* (§1.6.1, §1.9 — against *standardised* in §1.5.4), *internationalization* (§1.6.1), *recognized* (§1.7 end — against *recognise* in §1.3.2). Also American punctuation inside closing quotes in §1.3 ("containers," and ``echo.") where British convention places it outside.

13. Citations. The Kowalski epigraph needs a source (it is from *Logic for Problem Solving*, if memory serves — worth verifying). The Berners-Lee 1998 quotation should cite the "Webizing Existing Systems" Design Issues note. "Penguins (Lager & Wielemaker)" should be a proper `\cite`. And the Yogi Berra epigraph is famously of disputed attribution (variously assigned to Berra, van de Snepscheut, and others); Springer copy-editors may query it — "attributed to Yogi Berra" is the safe form. Finally, the hardcoded "Chapter 4," "Section 4.1/4.2," "Chapters 2–6," "Figure 1.3," etc. are maintenance hazards even setting LaTeX aside; convert to `\ref`s in the same pass.

Readability and register

14. Three register lapses. "the knowledge that these agents need to do their thing" (§1.1) is well below the surrounding register. Proposed: "Expressed more fully, the Prolog Web is populated -- and indeed sustained -- by Prolog agents, and it supplies, in the form of distributed Web Prolog databases, much of the knowledge on which those agents rely." Second, "The significance of this move should not be understated" (§1.5.2) — the idiom is contested; "should not be underestimated" is unimpeachable. Third, in §1.5's opening, "an adaptation to the Web of a profile of the Prolog language (Web Prolog)" stacks prepositional phrases awkwardly; "a profile of the Prolog language adapted to the Web (Web Prolog)" reads cleanly.

15. Two small transitions. §1.3.2 introduces `spawn/2` with no lead-in after the `echo_actor` listing; a half-sentence helps: "To bring this actor to life, we use `spawn/2`, which creates a concurrent process..." And in §1.7, "Regarding the history of Prolog, we do not suggest that nothing has happened between then and now" is an awkward pivot that also brushes against §1.4's claim that the book "does not discuss the history of Prolog." Proposed: "Nor was the story dormant between then and now: from the 1990s onward, Prolog has repeatedly adapted to the Web's changing landscape."

16. The demonstrator section is good as it stands. "The work has been done in cooperation with several AIs" is disarming and honest; keep it if you want the transparency, or formalize to "with substantial assistance from AI programming assistants" if a copy-editor balks. The two-caveats paragraph is exactly right.

One last observation in the chapter's favor: the "Finally, a note on scope" paragraph closing §1.5 is among the strongest in the chapter — three crisp negations that inoculate against the three most predictable misreadings. If you adopt the split proposed in point 4, make sure it remains the closing note of the motivation section rather than getting stranded mid-subsection.

Happy to draft the expanded §1.4 paragraph on FYPB's documented weaknesses, or the agent footnote in full, if you'd like to see them worked out.

You Message 3

2 Jul

Re: Happy to draft the expanded §1.4 paragraph on FYPB's documented weaknesses, or the agent footnote in full, if you'd like to see them worked out.

Yes, please do.

Claude Message 4

Here are both, in splice-ready form.

1. The expanded §1.4 paragraph

Placement: immediately after the paragraph introducing FYPB and PN50Y ("...which followed a year later. As they are written by...taken as probably the best and most current descriptions...") and before "What makes FYPB and PN50Y suitable as background...". That way the papers are introduced, their diagnosis is summarized, and only then do you explain why they matter to your project.

latex

The picture these papers paint is candid. Among the weaknesses they document, four are of particular relevance here. First, portability between Prolog systems is limited: the ISO standard covers only a core of the language, and features that real applications depend on -- modules, tabling, constraints, foreign-language interfaces, even such mundane matters as reading a file -- differ from system to system, so that non-trivial programs rarely run unchanged on more than one implementation. Second, and partly as a consequence, the ecosystem is fragmented: a dozen or so actively maintained systems, each with its own extensions, libraries, and user base, and no shared mechanism by which their strengths can be combined. Third, the tooling and library ecosystem lags well behind what developers of mainstream languages take for granted, raising the cost of entry for newcomers. Fourth, the computing landscape has shifted beneath the language: Prolog systems were shaped by the assumptions of the desktop era, and the authors identify a threat in what they call the "post-desktop world of JavaScript web applications" -- a world of browsers, mobile devices, and networked services in which Prolog has so far been a marginal presence. These four concerns -- portability, fragmentation, tooling, and the Web -- will recur throughout this book, and the Trinity should be read, in part, as a considered response to each of them.

Two notes on this draft. First, I have written the four concerns from my recollection of FYBP's actual discussion (the ISO core's limited coverage, system-specific extensions, the tooling gap, the post-desktop remark), but you have the paper and I don't in front of me — please verify that each attribution holds and that "post-desktop world of JavaScript web applications" is verbatim, since it appears in quotation marks both here and in your existing §1.5.4. If any of the four is more your gloss than their claim, soften with "as we read them" rather than dropping it. Second, the closing sentence is doing deliberate work: it sets up §1.5.4's "Where FYPB sees weakness in X, we see Y" contrasts, which currently arrive without preparation. With this paragraph in place, §1.5.4's opening "As we learned above..." now genuinely refers back to something learned.

If the paragraph feels too long for the section's rhythm, the cuttable material is the parenthetical examples in the first concern ("modules, tabling, constraints...reading a file") — but I'd keep them, because they are what makes "limited portability" concrete rather than a slogan.

2. The agent footnote

Placement: attached to the first occurrence of "Prolog agents" in the second paragraph of the chapter, replacing the current parenthetical "(The term ``agent" is used very liberally throughout this book and does not imply agency in its fullest sense.)".

latex

```
\footnote{A word on terminology. The term ``agent'' is used throughout this book in a deliberately modest, architectural sense: an agent is a computational process with a persistent identity, private state, and the ability to communicate with other such processes by exchanging messages. No commitment is implied to agency in any stronger sense -- neither the beliefs, desires, and intentions of the classical agent-theoretic literature, nor autonomy, goals, or anything approaching a mental life. Nor should the term be read through the lens of its most recent usage, in which an ``agent'' is a large language model equipped with tools and a control loop. Prolog agents as defined here are far simpler creatures. That said, the modest sense is designed to be a substrate for the stronger ones: an entity that persists, remembers, reasons over a private knowledge base, and converses is precisely the kind of entity out of which more ambitious agents can be built, and Chapter~\ref{sec:prolog-ai-agents-on-the-agentic-web} makes contact with both the classical and the LLM-flavoured notions deliberately.}
```

Three notes here. First, the final sentence turns the disclaimer into an asset — the footnote doesn't merely retreat from the loaded word but explains why the modest sense was chosen, which preempts the reviewer question "why call these agents at all rather than processes?" If you prefer a pure disclaimer, cut everything from "That said" onward; the footnote still works, just more defensively. Second, if you want to anchor "the classical agent-theoretic literature," the natural citation is Wooldridge and Jennings' 1995 survey (*Intelligent Agents: Theory and Practice*, Knowledge Engineering Review) — I've left it uncited since I don't know your bibliography keys, but it's the standard reference for the BDI-flavoured sense and would signal to AI-community readers that the choice is informed rather than naive. Third, check the `\ref` target: I've guessed `sec:prolog-ai-agents-on-the-agentic-web` from your §1.9 labels, and note that the main text near this footnote should retain a brief in-line signal too — I'd keep something minimal like "(in a sense to be qualified shortly)" or simply let the footnote marker do the work, but don't leave the sentence asserting "capable of reasoning, communicating, and coordinating" entirely unhedged in the body if the footnote is the only qualification, since footnotes are skippable.

One consequential follow-on: once this footnote exists, the phrase "and the simplest type of agent" in §1.3.1 and "nodes are themselves agents" in §1.5.2 both become consistent with a stated definition rather than with an unstated intuition — which is exactly what point 7 of my earlier critique asked for. If you also adopt the one-sentence taxonomy in §1.3.1, the footnote and that sentence should be checked against each other for wording (both use "actor" and "node" as the two primary kinds; keep the umbrella term "agent" in both).

You Message 5

2 Jul

Should I upload the images so that you can look at them too?

Claude Message 6

Yes, that would be genuinely useful — several of my critique points touched on figures I could only assess through their captions and the surrounding prose. With the images in hand I could check things like:

Whether **Figure 1.1 (trinity)** actually shows the "mutual dependencies" the text leans on — the prose says the arrows indicate "many interdependencies and symbiotic relationships," and whether the diagram carries that or just shows three labeled circles matters for how hard the text should work.

Whether **Figure 1.3 (prolog-web-intro-10)** visibly shows the two "containers" per node that §1.3 describes, and whether the emoji-actors, domain labels, and client connections are legible at the print width — this figure is doing the heaviest expository lifting in the chapter.

Whether **Figure 1.4 (an-actor)** shows all five anatomical features the prose enumerates (pid, mailbox, receive/send, spawn, private clause database), and in the same order the text walks through them.

Whether the **mindmap (summary-diagram-first)** actually matches the six-part structure of §1.9 — mindmaps drift out of sync with restructured chapters more often than any other figure type, and yours has "expanded the exposition of the three most important aspects" as its stated job.

Whether the **demonstrator screenshot (portal-o)** shows the workspace/terminal/Examples/Logger layout that §1.8 enumerates, or whether the UI has evolved past the screenshot.

The two I'd deprioritize: the spider-web chapter opener is decorative, and for the **inventors montage** the main issue isn't visual but legal — Springer will want documented rights or permissive licenses for each photograph of the five people, which is worth sorting now rather than at production. I can't verify licensing from the image itself, but I can at least check that the left-to-right order matches the caption and the prose walkthrough.

A couple of practical notes: upload them at the resolution you'll submit, since I can then also check pixel dimensions against the `\includegraphics` widths for print sharpness (a 6 cm figure wants roughly 700+ pixels across at 300 dpi). And if any of the diagrams exist as SVG or PDF source rather than PNG, those are the better versions to eventually submit anyway — worth checking whether `trinity` and `an-actor` were born vector.

Send whichever subset you like; Figure 1.3 and the mindmap are the two where a look is most likely to change my advice.

Attachment hidden in shared chat The shared page does not expose the uploaded source file or image.

2 Jul

Claude Message 7

These two were the right ones to send — both change my advice, and Figure 1.3 turns out to have some genuine problems that a careful Prolog reader will catch.

Figure 1.3 (prolog-web-intro-10)

1. The code at n4.org is broken as shown, and this is the most important finding. The visible clauses are:

prolog

```
p(X) :- q(X), r(X,Y).
r(X,Y) :- rpc('http://n1.org', p(Y)).
```

Three problems: `Y` is a singleton in the first clause, `X` is a singleton in the second, and — worse — no `q/1` facts are visible at n4.org, so `?- p(X)` there would raise an existence error. Your target audience will read this code, and singleton variables in a figure of a Prolog book are the kind of thing FYPB's authors would circle in red. The fix also improves the figure's pedagogy, because this node is your only illustration of the "nodes can be clients too" parenthetical. I'd replace with:

prolog

```
p(X) :- q(X).
q(X) :- rpc('http://n1.org', p(X)).
```

No singletons, everything defined, and the node-as-client role is unmistakable. Then add one sentence to the prose pointing at it: "Note that the second clause at n4.org makes that node a *client* of n1.org — nodes can be clients too."

2. The figure's echo actor disagrees with the text's echo actor. The text defines `echo_actor/0` (the sender's pid travels in the message, replies are wrapped in `echo(Msg)`); n13.org shows `echo(Pid)` (interlocutor fixed at spawn time, raw `Input` echoed back, plus a `stop` case). Both are idiomatic, but a reader who cross-references — and the text explicitly invites them to, since the echo actor is the chapter's flagship example — will find two different programs under one name-family and wonder which is canonical. Harmonize in one direction or the other; changing the figure to the text's `echo_actor` is the smaller intervention.

3. Unexplained smiley in the browser terminal. The second browser window contains a smiley, but the prose establishes smileys as "Prolog actors of various kinds that live and work at nodes." If this smiley foreshadows the browser-node architecture (Appendix C territory) or represents the client's toplevel actor, Chapter 1 doesn't say so, and it quietly breaks the visual vocabulary the reader has just been taught. Either remove it or spend a sentence: "the smiley in the second terminal hints at something we develop much later: actors can live in browsers too."

4. The two-containers order. In every node box, the program/data container is on *top* and the actor container *below*, but the prose lists them the other way ("one container dedicated to the management of actor processes, the other to the storage..."). Trivial fix in the prose: reverse the order so text and eye agree.

5. The off-frame arrows are good but unexplained. The thin arrows entering from beyond the rounded frame nicely convey that this is a fragment of a larger network — exactly what the text claims — but nothing tells the reader they're deliberate rather than a cropping accident. One clause in the caption fixes it. Note also the arrow-weight inconsistency: node–node and client–node links are thick double-headed arrows, while the $n_4 \leftrightarrow n_{41}$ connection is drawn as two thin single-headed arrows, the same style as the off-frame arrows. Unify: thick double-headed for in-frame connections, thin for off-frame continuations.

6. n22.org and n1.org have identical databases. If this illustrates replication or caching, nothing signals it; as it stands it reads like a copy-paste slip. Either vary one of them or, if replication is the point, say so.

7. Good news: the figure earns its keep. The terminal showing `X = a ; X = b ; X = c .` is your backtracking-as-pagination claim made visible; the client roster (two browsers, headset, robot) matches the prose enumeration exactly; and the `rpc` clause supports the node-as-client point once repaired. The figure and text are unusually well coupled — which is precisely why the small disagreements above matter more than they would in a decorative figure.

8. Production concerns. The resolution is comfortably sufficient at `\textwidth`, but the code font inside `n4.org` and especially `n41.org` will render at roughly 4–5 pt in print — below comfortable legibility. Consider enlarging the code relative to the boxes, or stripping `n41.org` down to smileys only (its code adds nothing the other nodes don't already show). Separately: the Oculus headset (with visible logo) and the Furhat photograph are product photos of identifiable commercial hardware; Springer will ask about permissions and trademark clearance. Replacing both with simple line-drawn icons sidesteps the issue entirely and would also unify the figure's visual style, which currently mixes diagram and photograph.

9. Proposed caption, folding in the partition→fragment fix from my earlier point 9:

latex

```
\caption{A small fragment of the Prolog Web. Nodes (boxes) store Web
Prolog programs and data (upper container) and host actor processes
(lower container, shown as smileys). Clients -- here two browser-based
terminals, a VR headset, and a social robot -- connect to nodes, and
nodes connect to one another; the second clause at \texttt{n4.org}
shows a node acting as a client of another node. Arrows leaving the
frame indicate that the network continues beyond what is shown.}
```

Figure 1.10 (summary-diagram-first)

10. The map is faithful to Parts I–III and silent on Parts IV–VI — so the prose over-claims. "The mindmap reflects the structure of the rest of this book" is not quite true: Web Prolog's ancestry (Chapter 2), the agent taxonomy (Chapters 3, 5, 6), and the Prolog Web's neighbours (Chapters 4, 7, 8) are all present, but there is no trace of the paradigm chapter, the two community chapters, or standardization. Extending the map would overload it; adjust the prose instead:

latex

```
The mindmap charts the conceptual terrain developed in
Parts~I--III. The remaining parts step back from this terrain: to ask
what paradigm is taking shape (Part~IV), what the proposal offers to
various communities (Part~V), and what would be required to move from
vision to execution (Part~VI).
```

11. The arrow semantics need a key. The map mixes three relation types: labelled solid arrows pointing outward from the core ("extends," "borrows a lot of ideas from"), unlabelled solid arrows pointing inward (the is-a hierarchy: shells → toplevels → actors → Prolog agents; nodes → Prolog agents), and unlabelled *dashed* arrows pointing inward ("MASs & ECAs" → Prolog agents; "the Agentic Web" → the Prolog Web). The dashed ones are genuinely undecipherable: is the Agentic Web an influence on, an application of, or a habitat for the Prolog Web? A reader cannot recover the relation, and the direction is arguably backwards if the intent is "Prolog agents are building blocks *for* MASs and ECAs." Cheapest fix is a caption sentence: "Unlabelled solid arrows denote subtype relations; dashed arrows indicate looser, forward-looking connections taken up in Chapters 7 and 8." Better still, label the dashed arrows in the artwork itself.

12. MAS is used before it exists. ECA is glossed in the chapter's second section, but "MAS" appears nowhere in the Chapter 1 text — its first occurrence in the whole book may be this figure. Expand in the caption ("multi-agent systems (MASs) and embodied conversational agents (ECAs)") or, better, write it out in the artwork, where "MASs" is in any case an awkward plural-acronym.

13. One genuine find: "extension of a subset of" is better than the text's own definition. The map characterizes Web Prolog as an "extension of a subset of" Prolog — which is more precise than the body text's "carefully crafted profile of the Prolog language," and it dissolves half of the profile-polysemy problem I raised in point 10 of the first critique. I'd import it into the opening paragraph: "a carefully crafted *profile* of the Prolog language — more precisely, an extension of a well-chosen subset — reimagined as..." Now the figure and the text define the language identically, and "profile" gets an immediate gloss.

14. Production: this figure must not ship as a JPEG. It is line art with text — JPEG compression puts halos around every glyph, and Springer production will bounce it or, worse, print it as-is. Export as PDF or SVG from the original tool (this looks Keynote- or Draw-born, so vector export should be one click); PNG is the acceptable fallback. The same check is worth running across all figures: `trinity`, `an-actor`, and this one were almost certainly born vector and should be submitted that way, with PNG/JPG reserved for the screenshots and photographs.

15. Verify triangle consistency with Figure 1.1. The map's core — the three names joined by black double-headed arrows — is presumably a miniature of the trinity figure. If the two differ in arrow style, ordering, or orientation, readers will subconsciously register the mismatch; if they agree, the map reads as "Figure 1.1, elaborated," which is exactly the effect you want. I can't check without seeing `trinity`, so eyeball them side by side.

The overall verdict on both: strong figures with high text–figure coupling, let down by fixable code bugs (n4.org), one vocabulary breach (the browser smiley), unexplained arrow semantics (mindmap), and format/permissions issues that are better resolved now than during production. If you send the `trinity` and `an-actor` figures, I can close out points 7 (actor anatomy completeness) and 15; otherwise, happy to redraw the n4.org code block and caption text in whatever form your figure source uses.

Attachment hidden in shared chat The shared page does not expose the uploaded source file or image.

2 Jul

Claude Message 8

These two are in much better shape than Figure 1.3 — mostly a matter of closing loops and a couple of optional improvements.

Figure 1.1 (trinity)

1. Point 15 closes positively. The mindmap's core triangle is, for all practical purposes, this figure reproduced verbatim: same layout (Web Prolog apex, rotated labels on the lower edges), same black double-headed arrows, same typography. Readers will register the echo, consciously or not, and it does exactly the "Figure 1.1, elaborated" work I hoped for. Worth cashing in explicitly with one clause in the mindmap's caption: "The triangle at its centre reproduces Figure~\ref{fig:the-big-picture-4}, now embedded in its wider conceptual context." That turns an implicit visual rhyme into a navigational aid.

2. The figure shows the triad but not the trichotomy. The prose introducing it says the three components are "three different kinds of entities, playing three distinct roles" — but the figure gives no hint of what those roles are, and §1.5 later articulates them crisply: language, execution model, infrastructure. Consider adding a small-type gloss beneath each label:

- Web Prolog — *a language*
- Prolog agents — *an execution model*
- the Prolog Web — *an infrastructure*

This costs nothing visually (the figure is spacious at 5.5 cm) and it plants §1.5's summary trichotomy on page 2, so that when the reader reaches "Web Prolog provides a language for expressing... Prolog agents provide an execution abstraction... the Prolog Web provides an infrastructure...", the sentence lands on prepared ground. If you prefer to keep the figure maximally austere — and the text does say it shows "the highest level of abstraction we shall care about" — then it's defensible as is; the arrows-as-generic-interdependency reading is honestly declared in the prose. But the glosses are the better trade, I think.

3. Minor. The figure is generously sized in pixels for 5.5 cm, so no print concern — but it is another born-vector PNG; same PDF/SVG export advice as before.

Figure 1.4 (an-actor)

4. Point 7 closes positively. All the anatomical features the prose enumerates are present: the pid (123456, under the title), the mailbox (with queued envelopes), receive (mailbox → process arrow), send (twice — an outbound message leaving the boundary, and a directed send to a known peer), spawn (to a fresh smiley), and the private Prolog database (dotted box). The count also checks out: the text promises three actors, "one of which has just been spawned and one that has been opened up," and the figure delivers exactly that — the opened actor, the spawned child on the right, and the message-recipient peer on the left completing the trio. Figure and text are tightly coupled.

5. One genuine text–figure tension: is the process the actor, or inside the actor? The prose's first sentence says "A Prolog actor is a process," but the figure draws the actor as a large circle *containing* a smaller circle labelled "Process" — making the process one component among several, alongside the mailbox and database. Both framings are defensible (the containment view is standard in Erlang pedagogy), but they shouldn't disagree with each other in adjacent column-inches. The cheaper fix is in the prose:

latex

```
\noindent A Prolog actor is organised around a \textit{process} -- a
thread of control capable of communicating with the world around it
through messaging.
```


The dearer fix is relabelling the inner circle to something like "thread of control" or "behaviour" and keeping the prose as is. I'd take the prose fix.

6. The figure quietly encodes asynchrony — spend a sentence on it. Notice that the incoming message arrow terminates at the *mailbox*, not at the process, while *receive* is a separate, actor-initiated step. That is the single most important operational fact about the actor model — sends are fire-and-forget, and the actor consumes messages at its own pace — and the figure gets it exactly right without the text pointing at it. One sentence after the mailbox/receive enumeration extracts the value: "Note that, as the figure shows, incoming messages are delivered to the mailbox, not to the process itself: sending is asynchronous, and the actor retrieves messages with `\texttt{receive}` at a time of its own choosing." Erlang-literate readers will already know this; everyone else needs it, and it foreshadows the blocking-surface discussions later in the book.

7. Three small visual points. (a) The mailbox is drawn as a fixed row of slots, two filled, three empty — which could be misread as a bounded buffer, when mailboxes are unbounded queues; if the artwork is easy to touch, an open-ended right edge (or a trailing ellipsis slot) removes the ambiguity, but this is minor. (b) Dashing is used for two different things — the actor's boundary and the database box — with the mailbox drawn solid; harmless, but if the dashed database box was meant to signal "private/encapsulated," note that the same dash on the outer boundary dilutes that signal. (c) The blue envelopes and khaki fill will survive greyscale conversion fine if the book prints in black and white, so no action needed there — just worth knowing you're safe.

8. Consistency win worth preserving. The smileys here mean exactly what they mean in Figure 1.3 — whole actors seen from outside — and the opened actor is what a smiley "contains." That's precisely the visual vocabulary the prose sets up ("Hiding an actor behind an emoji makes it look simpler than it actually is"). Whatever you do when repairing Figure 1.3, keep the smiley semantics untouched; these two figures currently teach it correctly, and the only breach in the whole set remains the unexplained browser smiley from point 3 of the previous round.

So the full-figure audit for Chapter 1 nets out as: Figures 1.1, 1.4, and the mindmap are sound, needing only caption/prose adjustments and vector re-export; Figure 1.3 needs the `n4.org` code repaired, the echo-actor harmonized with the text, and the photograph-permissions question settled; and the inventors montage remains a rights question rather than a content one. If you fix the `n4.org` clauses in the figure source, send the revised version and I'll re-check it — that's the one change where a second look is warranted.

You Message 9

2 Jul

Attachment hidden in shared chat The shared page does not expose the uploaded source file or image.

Let's move on to ch. 2.

Claude Message 10

Chapter 2 read in full. The verdict up front: this is a stronger chapter than Chapter 1 in its second half and a weaker one in its first. The progression of the concurrency material — primitives, then receive semantics, then worked examples, culminating in "Getting answers through backtracking" and the toplevel sketch — is genuinely well-built, and the guards-that-reason section (§ Guards) plus the backtracking-under-external-control section are the best writing I've seen in the manuscript so far. The problems cluster in three places: the front matter (§2.1–2.2) is baggy and register-uneven; there are several **code-level bugs and semantic gaps** in the examples that your target audience will find; and the paradigms figure has both a rights problem and a content mismatch with the text.

Structure

1. The §2.4 roadmap promises content the chapter doesn't deliver. The section opener announces "a count server, a fridge simulation, a universal stateful server with hot code swapping, actors playing ping-pong, and an approach to building simple supervision hierarchies." The chapter contains no hot-code-swapping server and no supervision hierarchies — those presumably migrated to Chapter 5 during restructuring — while the two sections that *are* the chapter's highlights (event-driven state machines, answers through backtracking) go unannounced. Splice-ready replacement:

latex

In this section we shall look at a count server, a fridge simulation, a technique for hiding message protocols behind predicate APIs, delayed sending, actors playing ping-pong, event-driven state machines whose transitions consult the knowledge base, and — most importantly — a demonstration that Prolog's backtracking search can be placed under external, message-driven control. They are all concurrent programs, but only locally so. (Distributed programming is dealt with in Chapter~\ref{sec:the-prolog-web}, and reusable behaviours such as generic servers and supervisors in Chapter~\ref{sec:implementing-prolog-generics}.)

2. The deferral definition is wrong by its own letter — and the priority queue depends on the correction. §Messages deferred defines: a message is deferred if it "does not match any of the patterns." But in the priority-queue example, `7-low` *does* match the pattern `Priority-Message`; only the guard `Priority > 10` fails — and the example works only if such messages are also deferred. As written, the definition doesn't cover the case your own flagship example relies on. Fix at the definition:

latex

```
If no receive clause \emph{accepts} a message in the mailbox -- that is,
if for every clause either the pattern fails to match or the guard fails
-- the message is \textit{deferred}: it remains in the process's mailbox,
possibly to be handled by a later \texttt{receive/1-2} call.
```

3. The guard's once-semantics is stated 500 lines too late. The crucial fact that a guard is evaluated for its *first solution only* appears in the state-machines section ("though only its first solution is considered"), not in §Guards where guards are introduced and celebrated. The husband/wife example invites exactly the question the placement leaves open: what if the knowledge base gave `husband(xantippa, H)` two solutions — does the clause commit to the first, and can failure of the *body* re-try the guard? State it at introduction: "A guard is evaluated as if wrapped in `\texttt{once/1}`: the clause commits to the guard's first solution, and failure in the body does not re-satisfy it." (Assuming that is the semantics — if body failure *can* backtrack into the guard, that's a different and more radical design that needs its own paragraph.) Also worth a sentence on what happens if a guard throws.

4. The shell is never defined, and its name drifts. §2.3.1 opens "we may want to talk to a Prolog *shell*" with no definition, no relation to "toplevel" (which Chapter 1's mindmap places above shells in the taxonomy), and then the count-server session suddenly calls the same entity "the toplevel process." Since the shell-is-an-actor insight is one of the book's distinctive moves (my Chapter 1 point 6), plant a definition at first use:

latex

```
If we need to hold a Prolog-style conversation with an actor, we talk to
a Prolog \textit{shell}: the actor that sits behind an interactive
terminal, reads our queries, and prints the answers. A shell is a
specialised kind of \textit{toplevel actor} -- a distinction developed in
Chapter~\ref{sec:prolog-agents} -- and, crucially, it is an actor like
any other: it has a pid, a mailbox, and a private database, as the
examples below will exploit.
```

Then replace "the toplevel process" in the count-server session with "the shell."

5. The LP-landscape survey is a \subsection doing a \subsection's job in the wrong section. Structurally it hangs alone off §2.1; substantively it's a survey of ALP/ILP/Datalog/WFS/ASP/PLP/CLP/LPS justified by a promissory note ("will reappear in later chapters"). Two options: promote it to a subsection and keep it, tightening the forward reference to a concrete \ref; or move it to the chapter where the accommodation claim is cashed (Chapter 10 or wherever "any system that can accept a goal and return answers" is developed). If it stays, fix "ProbLog is a Python-based toolbox" — ProbLog is a probabilistic logic programming *language* whose reference implementation happens to be in Python; the current phrasing miscategorizes it.

6. The scripting-language subsection is the chapter's flabbiest stretch. The long JavaScript paragraph is an encyclopedia-style feature recitation ("dynamically typed, interpreted multi-paradigm... callbacks, promises, and async/await... JSON, is lightweight and widely used") that sits well below the surrounding register and says nothing a reader of this book doesn't know. Cut it to three sentences: JS began as browser glue, became a general-purpose full-stack language via Node.js, and now defines what "web programming language" means in practice. Second, the subsection "Is Web Prolog suitable for programming the Web?" is a two-sentence stub — fold it into the paragraph preceding the four-point checklist and drop the heading. Third, the four-point checklist promises four discussions but delivers three-and-a-stub; after the fold, renumber to match.

Correctness of examples and semantics

7. The `link(false)` example is broken — the code contradicts both its own output and the surrounding text. The displayed query is `?- spawn(spawn(receive({})), Pid, [link(false)])`, i.e., the option is on the *outer* spawn. But the prose says "the child (created by the inner call to `spawn/3`) does not necessarily terminate if the parent (created by the outer call to `spawn/1`) terminates" — labelling the *inner* call as `spawn/3` and the *outer* as `spawn/1`, the reverse of the code. And the output `(Alive = [91033267, 24917621])`, inner actor surviving) is only consistent with the prose's version: with the option on the outer spawn, the inner actor was spawned by plain `spawn/1` with the default `link(true)` and should have died with its parent, exactly as in the first example. The text describes the right experiment; the code shows the wrong one. Fix:

latex

```
?- spawn(spawn(receive({}), _, [link(false)]), Pid).
Pid = 10023812.
?- actors(Alive).
```

```
Alive = [91033267, 24917621].
?~
```

8. Flag the two deviations from Erlang's linking at first introduction, not a hundred lines later. Web Prolog's `link` is (a) unidirectional where Erlang's is bidirectional, and (b) on by default where Erlang's `spawn` does not link at all (`spawn_link` does). Both deviations are acknowledged — but only at the end of "A hierarchy of actors," after an Erlang-literate reader has processed four examples with the wrong mental model. One forward-flagging sentence at the definition ("Readers who know Erlang should note two deliberate departures here, justified in Chapter~4: the dependence is one-way, parent to child, and it is the default") costs little and prevents the misreading. Given how central this containment discipline is to Chapter 4's client-owner story, it deserves the early flag.

9. Pid hygiene across the chapter's sessions. Pids are introduced as unforgeable random integers, yet `60367387` is the pid of the `2 > 1` actor in the monitoring example *and* of the count actor in §2.4.1 — an exact collision that reads as copy-paste, not randomness. Meanwhile the shell's own pid drifts silently: `85234512` in §2.3.1, `91033267` in the `actors/1` examples ("We can infer that this must be the shell" — the reader who remembers `85234512` will infer the opposite), `41167597`, `34871244`, `97216744`, `90054377` later. Either regenerate all example pids to be globally unique across the chapter with one consistent shell pid, or add an early footnote: "The transcripts in this book are drawn from many different sessions; the shell's pid therefore varies between examples." The `actors/1` inference sentence needs that footnote or it actively misleads.

10. The down-message vocabulary has a hole: goal failure. The chapter documents `Reason = true` for normal termination, error terms for crashes (`existence_error(proc,q/1)`), and arbitrary terms via `exit/1-2`. But `spawn(2 > 2, Pid, [monitor(true)])` — what arrives? Presumably `down(_, _, false)`. Failure is a termination mode unique to a *relational* actor language — Erlang has no analogue — so it's precisely the case this book should document rather than skip. One example or one sentence closes the gap, and it foreshadows the toplevel's `failure` message nicely.

11. Monitor phrasing: the runtime notifies, not the child. "monitor(true) instructs the child actor to send a down message to its parent just before terminating" is the wrong agent — a child killed by an exit signal or a thrown error sends nothing; the runtime delivers the notification (as your own `existence_error` example demonstrates). Rephrase: "instructs the runtime to notify the parent with a `{down}` message when the child terminates, for whatever reason." The section's opening sentence already has the correct framing; the example paragraph contradicts it.

12. Name registration cleanup is relied on but never stated. The sequence kills the registered `echo_actor` and then shows that sending to the `name` raises an error — which only works if registration is automatically removed on death (as in Erlang). Say so: "When a registered actor dies, its name registration is removed with it." Also worth one clause on the earlier registration example: can a name be registered twice, and what happens if you try?

13. Two subtleties in `simple_toplevel` deserve a sentence each. First, `spawn(session(Pid, Self), Pid, Options)` passes an unbound `Pid` inside the goal and expects `spawn`'s binding of its second argument to be visible in the copied goal — this works only if `spawn` is defined to generate the pid, bind it, and *then* copy the goal to the new process. That ordering guarantee is implementation-critical (anyone porting this to SWI threads will hit it) and clever enough to point at: "note that the goal mentions the very pid that `spawn` is about to assign; `spawn` guarantees the binding is made before the goal is transferred." Second, `'call'`, `'call'`, `'call'`, `'next'`, `'$stop'` use dollar-prefixed *atoms* two pages after the shell's dollar-prefixed *variable* notation was introduced; one disambiguating clause ("the leading `$` here is mere naming convention for protocol-internal messages, unrelated to the shell's dollar notation") prevents a real confusion.

14. Where does an actor's output go? The ping-pong actors, the alarm, and `flush/0` all call `format/2` from spawned processes, and the output appears at the shell's terminal — but nothing says why. Erlang solves this with group leaders; Web Prolog presumably routes output to the owning console somehow, and this is a genuine design point, not a triviality (it matters enormously once actors are remote, in Chapter 4). One paragraph, or a footnote deferring to the node architecture chapter. Relatedly, the ping-pong transcript showing `true.` *before* the actors' output lines is a lovely authentic detail — spend one sentence on it ("note that the shell's `{true}` appears first: the query merely spawned the actors and succeeded immediately; their output arrives asynchronously") so beginners don't read it as a typo. And a small one: `flush/0` hard-codes the string "Shell got", which lies if the utility is ever called from a non-shell actor — either scope the claim ("a shell utility") or parameterize the label.

15. The FYPB feature list is reproduced verbatim — treat it as the quotation it is. "Below, we reproduce their list of features" followed by twelve items lifted from the paper is honest attribution but not quotation formatting; for a Springer book, either set the list as a block quotation with the citation, or paraphrase it into your own words (which would also let you fix the label drift: the text first says essential vs. "important (yet non-essential)", then calls items 7–12 "less essential" — FYPB's own term is "important," so use it both times). Two adjacent claims also need attention: "no other community... has found itself in the position of having to determine what should be considered a *real* language of this kind" will draw a referee's counterexample (the Lisp world's Common Lisp/Scheme/dialect wars are exactly this); soften to "few other communities." And "For almost thirty years now" — the standard is from 1995, so it's now *more than* thirty; write "for three decades" and it stays true.

Terminology and consistency

16. "Superset of a subset" vs Chapter 1. Chapter 1's mindmap says "extension of a subset of" Prolog and (per my earlier point 13) that formula was worth importing into Chapter 1's prose. Chapter 2 opens §2.2 with "superset of a subset of the ISO Prolog core standard." Same idea, two formulas — pick one ("an extension of a well-chosen subset" is my preference; "superset" invites the pedantic reading that any extension is a superset) and use it in the mindmap, Chapter 1, and here.

17. "Trinity Prolog demonstrator" vs "Trinity demonstrator." Chapter 1 establishes the latter; §2.3's preamble says the former. One name.

18. The British/American pass this chapter badly needs. American stragglers: *behavior(s)* (at least eight instances, including the section title "The toplevel behavior" and the Summary), *standardized* (twice, against *standardised* in the epigraph and elsewhere), *characterizes/categorize* (against *characterise* later in the same section), *serialization*, *specialized*, *gray* (ping-pong caption), *cancellation* (also misspelled by British lights: *cancellation*), *recognizably* (Summary). Plus *unforgable* → *unforgeable* regardless of dialect.

19. Citations and quotations bundle. (a) The second O'Keefe epigraph ("Prolog is different, but not that different") has no source — it needs one or must go. (b) The Hébert dark-room epigraph is from *Learn You Some Erlang*, which you cite later for the priority queue; attach the citation to the epigraph too, and note that Springer generally asks authors to clear epigraph permissions regardless of length. (c) The Larry Wall obscenity quote needs source verification — it riffs on Potter Stewart and circulates in several forms; if it can't be pinned to a citable source, attribute as "attributed to." (d) You appear to have *two* bibliography keys for Armstrong's 2003 thesis (`Armstrong2003` and `armstrong2003concurrency`) — check for a duplicate entry. (e) The YouTube footnote URL `watch?v=K8nxTSPHZh5` has a 12-character video ID; YouTube IDs are 11 — the link is almost certainly broken. (f) The Prologue-for-Prolog predicate list (`maplist/2-8`, `countall/2`, etc.) is attributed to a specific source; verify each arity against that page before print, since an attributed list must match its source exactly. (g) Armstrong "(p.c. June 18, 2018)" → Springer style "(personal communication, 18 June 2018)".

20. Grammar and small slips. "predicates that gives a program access to the operative system" → "give... operating system" (an *operativsystem* calque worth catching before a copy-editor does); "predicates for constraint logic programming is listed" → "are listed"; "the people behind... SICStus Prolog appears" → "appear"; "Whether or not the points 1 and 2 applies" → "apply"; "it should now probably be described as general-purpose languages which also happens" → "as a general-purpose language which also happens"; "Here is how this might look like" → "might look"; "such abstractions turns out" → "turn out"; "good way around it" → "a good way around it"; "Javascript's" → "JavaScript's"; ping-pong caption: "the upper line represent... each gray bar represent" → "represents" twice. Also "Web Prolog is a very *general* special-purpose scripting language" — if the paradox is intentional, own it ("a special-purpose language of unusual generality"); as written it reads like a slip.

The figures

21. paradigms.png — a rights problem and a content mismatch. Three issues in ascending order of severity. *Resolution*: 1196 px across at `\textwidth` is roughly 250 dpi — marginal — but the real problem is that this is Van Roy's full poster, designed for A3; at Springer's ~12 cm text width the small box labels will be around 4 pt and effectively illegible. *Rights*: the caption's own placeholder "(Image source and license to be confirmed.)" must be resolved; the diagram is from Van Roy's "Programming Paradigms for Dummies" (2009) and your red rounded rectangles make this an *adapted* reproduction, which needs explicit permission ("Adapted from Van Roy (2009), with permission"). Van Roy has historically been generous about this diagram, so writing to him is the easy path. *But the better fix solves both problems at once*: redraw an **excerpt** showing only the three highlighted paradigms and the "+ concept" edges connecting them to their neighbours, with a caption directing readers to the full taxonomy in the cited paper. Legible, rights-clean (a redrawn excerpt with attribution is your own figure), and it focuses the reader on exactly what the argument uses.

22. The figure and the text disagree about which paradigm is highlighted — and the figure is right. Your second red box encloses "Imperative search programming" (SNOBOL, Icon, Prolog) — which is where Van Roy actually places Prolog — while the text says Web Prolog supports "*imperative programming* (also like in Prolog)." In Van Roy's taxonomy plain "Imperative programming" is a different, un-highlighted box (Pascal, C). The supporting paragraph then conflates two separate observations: assert/retract (named state — the imperative ingredient) and built-in backtracking search (the search ingredient). Those are precisely the two "+ concept" edges that jointly produce *imperative search programming*, so the fix is to let the text track the figure:

latex

```
This suggests that we can regard Web Prolog as a language that supports
three programming models: \textit{relational} & logic programming} (as in
Prolog), \textit{imperative search programming} (also as in Prolog --
this is where van Roy places Prolog once its dynamic database, a form of
named state, and its built-in backtracking search are taken into
account), and, as in Erlang, \textit{message-passing concurrent
programming}.
```

The following paragraph then already contains the right material (assert/retract, `nb_setval/2`, search "has little to do with logic") — it just needs its conclusion pointed at "*imperative search programming*" rather than bare "imperative programming." Also: check the capitalization of the name — he publishes as Peter **Van** Roy; the text and caption have "van Roy."

23. state-machine.png — code and figure agree; four small improvements. I verified the correspondence transition by transition: `e1[c1]/a1` from `s0` to `s1` ✓, the `e2` self-loop on `s0` ✓, the `e3` self-loop on `s1` ✓, the `*` fallback from `s1` to `s0` ✓. Improvements: (a) add an **initial-state marker** (the standard filled-dot-and-arrow into `s0`) — as drawn, nothing distinguishes the start state, and the code's entry point is `s0`; (b) the `*` wildcard is used without explanation — one caption clause: "the transition labelled `~*` fires on any other event"; (c) the `e(X) [c(X,Y)] /a(Y)` label notation is UML-statechart convention, which the prose walks through but never decodes as notation — one sentence ("we write `e[c]/ae[c]/a` for a transition taken on event `ee` `e`, provided guard `cc` `c` holds, performing action `aa` `a`") pays forward to Chapter 6; (d) at 840 px for a 9 cm figure you're at ~237 dpi — fine for this simple line art at screen, marginal in print; export vector.

24. The figure hides a genuine semantic asymmetry worth one paragraph. In state `s1` the catch-all `_AnyEvent` consumes every unmatched message; in state `s0`, which has no catch-all, unmatched events are *deferred* — they sit in the mailbox and may fire a transition arbitrarily later, when the machine happens to re-enter a state that matches them. So the same machine drops stale events in one state and queues them in another, and a queued stale event firing late is a classic statechart bug class. This is exactly the kind of "unexpected interaction between paradigms" your §2.2 subsection promises to be vigilant about, and it arises directly from the deferral semantics the chapter is otherwise proud of. Naming it here — even just "note that `s0` defers unknown events while `s1` discards them; the designer must choose deliberately, a matter we return to with SXML in Chapter 6" — turns a latent gotcha into demonstrated candour.

25. The word cloud (fig:wordcloud2) — I haven't seen it, but interrogate it. A word cloud in a Springer monograph invites the question: what does glyph size encode? If the answer is "nothing" or "aesthetics," a plain multi-column listing of the ISO built-ins (or a pointer to the appendix manual) serves the reader better and will survive review more comfortably. If size encodes something real (frequency in a corpus, say), the caption must say so. Send it along with the wordcloud, inventors, portal, and ping-pong-tasks figures if you want the same audit the others got.

26. Resolution roundup from the files you've sent (Springer text width ≈ 12 cm, target 300 dpi ≈ 1440 px): prolog-web-intro 2585 px ✓, mindmap 2918 px ✓ (but re-export as vector, not JPEG), trinity 1631 px at 5.5 cm ✓ generously, an-actor 1012 px at 5.6 cm ✓, paradigms 1196 px at `\textwidth` — marginal and moot if redrawn, state-machine 840 px at 9 cm — marginal, redraw vector.

Finally, what to protect in revision: the semi-deterministic-receive treatment (mailbox effects as non-backtrackable, "the same discipline Prolog programmers already accept for the dynamic database, now extended to the mailbox") is exactly the right theoretical move stated exactly clearly enough; the guards section's "the witness to that relation is delivered to the body as a binding" and the `if`-vs-`when` naming argument are the chapter's intellectual signature; and "Getting answers through backtracking" earns its buildup — the line about non-backtrackable sends being "the very thing that turns Prolog's search into an externally-driven answer stream" is the hinge on which the whole book's toplevel machinery turns, and it's placed and paced correctly. The revision job is to bring the chapter's first third and its example hygiene up to the standard its second half already sets.

Happy to draft the compressed JavaScript passage, the full corrected linking subsection, or a redrawn-excerpt spec for the Van Roy figure (which boxes, which edges, suggested caption) — say which.

You Message 11

2 Jul

Re: Happy to draft the compressed JavaScript passage, the full corrected linking subsection, or a redrawn-excerpt spec for the Van Roy figure (which boxes, which edges, suggested caption) — say which.

Do all three!

Claude Message 12

Here are all three, in order.

1. The compressed JavaScript passage

Placement: this replaces everything from "Over the years, the Web has become a key delivery platform..." down to the end of the Node.js paragraph. The paragraph beginning "It is clear that as a relational logic programming language..." stays and follows on naturally.

latex

```
\noindent JavaScript must be regarded as \textit{the} web programming
language of our times. It began, in 1995, as a small, domain-specific
scripting language confined to the browser, where its job was to add a
little dynamism to otherwise static pages. Three decades later it is a
general-purpose, multi-paradigm language -- dynamically typed,
event-driven, with mature support for asynchronous programming and, in
JSON, a native data-interchange format that the rest of the Web has
adopted wholesale. With Node.js it crossed over to the server side, so
that a single language now spans the entire stack -- an advantage whose
attraction for developers should not be underestimated, and one reason
JavaScript ranks among the most widely used programming languages on
Earth.
```


For our purposes, however, the details of the language matter less than the \textit{role} it plays.

Three notes. First, a catch that surfaced while doing this: the opening sentence of the current version — "Over the years, the Web has become a key delivery platform for a variety of sophisticated interactive applications in just about any conceivable domain" — appears *verbatim* in Chapter 1, §1.6 (the webizing section). Self-plagiarism across chapters is exactly the kind of thing a copy-editor flags; the compression removes it here, so no further action needed, but worth a global grep for other duplicated sentences. Second, the closing paragraph of the security subsection ("In summary, while JavaScript's initial role was confined to client-side scripting... its capabilities have expanded significantly. Today, it serves as a versatile glue language...") retells the same origin story a third time; with the compression in place, cut that summary paragraph down to its final sentence ("When designing Web Prolog... we must of course try to learn from such a popular and versatile tool, with a focus on the good parts in particular" — which also lands the *Good Parts* allusion better when it isn't preceded by throat-clearing). Third, the handoff sentence "For our purposes, however, the details of the language matter less than the role it plays" deliberately pre-empts the existing "What matters here is the role that JavaScript plays on the Web" two paragraphs later — when splicing, keep whichever of the two you prefer and delete the other.

2. The corrected linking subsection

This is a full drop-in replacement for §Linking, from the heading through the sentence about node owners. It fixes the buggy third example, states explicitly which spawn the option attaches to (the ambiguity that made the bug invisible), flags both Erlang deviations at the top per my point 8, and preserves your pids and the label.

latex

```
\subsection{Linking}\label{sec:linking}

\noindent A further spawn option, \texttt{link}, governs the lifetime
of the spawned actor. If the option is set to \texttt{true}, the life
of the new actor is tied to the life of its parent: should the parent
terminate -- for whatever reason -- the child is forced to terminate
too. In our proposal, the default value of this option is
\texttt{true}.

Readers who know Erlang should pause here, because this is a point at
which Web Prolog deliberately departs from its role model, in two
ways. First, Erlang's links are bidirectional, whereas the dependence
just described runs in one direction only, from parent to child.
Second, Erlang's \texttt{spawn} does not link at all -- linking
requires \texttt{spawn\_link} -- whereas in Web Prolog the tie to the
parent is the default and must be explicitly declined. Both departures
are motivated by the client--owner separation of the open Web, and both
are justified in Chapter~\ref{sec:the-prolog-web}; for now, we simply
demonstrate the behaviour.

In the following example, the outer call to \texttt{spawn/2} creates an
actor whose goal spawns a second, non-terminating actor and then
completes. When its goal completes, the outer actor terminates -- and
because the inner actor was created with the default
\texttt{link(true)}, it is automatically killed along with it:

\begin{verbatim}
?- spawn(spawn(receive({})), Pid).
Pid = 10782012.
?-
\end{verbatim}

\noindent Calling the built-in predicate \texttt{actors/1} confirms
that only one actor remains alive -- the shell itself:

\begin{verbatim}
?- actors(Alive).
Alive = [91033267].
?-
\end{verbatim}

\noindent The dependence does not run the other way. A parent whose
child terminates -- even abnormally -- is unaffected:

\begin{verbatim}
?- spawn((spawn(exit(die)), receive({})), Pid).
Pid = 40276924.
?-
\end{verbatim}

\noindent Even though the child killed itself immediately, calling
\texttt{actors/1} again shows that its parent \texttt{40276924} is
still alive:

\begin{verbatim}
?- actors(Alive).
Alive = [91033267, 40276924].
?-
\end{verbatim}
```



```

\noindent The link, then, makes the life of a child dependent on the
life of its parent, and never the reverse.

Since the option is on by default, declining it must be done
explicitly, by passing \texttt{link(false)} \emph{when the child is
spawned}. Note carefully where the option goes: it belongs to the
child's own spawn, and describes that child's relationship to its
parent. In the following example, it is the \emph{inner} actor that is
spawned with \texttt{link(false)}, and it consequently survives the
termination of the outer actor that created it:

\begin{verbatim}
?- spawn(spawn(receive({}), _, [link(false)]), Pid).
Pid = 10023812.
?- actors(Alive).
Alive = [91033267, 24917621].
?-
\end{verbatim}

\noindent The outer actor (\texttt{10023812}) is gone, its goal having
completed, but the inner actor (\texttt{24917621}) lives on -- now
without a living parent. Actors of this kind, deliberately detached
from the hierarchy that created them, are useful for long-lived
services, but they are also actors that no parent's termination will
ever clean up. For a reason explained in
Chapter~\ref{sec:the-prolog-web}, the distributed setting therefore
restricts them: clients are typically required to spawn with
\texttt{link(true)}, with \texttt{link(false)} reserved for node
owners.
\end{verbatim}

```

(Delete that stray closing `\end{verbatim}` on the last line — artifact of the block, not part of the text.) Two coordination notes: the following "A hierarchy of actors" subsection keeps its closing Erlang-comparison paragraph, which now reads as the fuller development of a contrast the reader was warned about, rather than a late surprise — you may want to soften its opening from "Erlang behaves differently here" to "As flagged above, Erlang behaves differently here." And the shell pid `91033267` in these transcripts still conflicts with the `85234512` established in §2.3.1 — this rewrite preserves your pids, so the session-continuity fix from my point 9 (footnote or renumbering) still applies on top.

3. The Van Roy excerpt: spec and mockup

The principle. A redrawn excerpt in your own layout and styling, expressing the taxonomy's factual relationships with clear attribution, is your own figure — the standard scholarly position is that facts and relationships aren't protectable, only the specific visual expression, which you are not copying. That said, a courtesy note to Van Roy costs nothing and Springer's permissions team may ask whether you sought one; I'd send it. Either way, the caption placeholder problem dissolves.

Which boxes (8). One shared root plus three descent paths, mirroring the full poster's left-to-right ordering (logic leftmost, message passing centre, state-rich rightmost):

1. *First-order functional programming* — the root, unhighlighted
2. *Functional programming* (Scheme, ML) — unhighlighted
3. *Deterministic logic programming* — unhighlighted
4. **Relational & logic programming** (Prolog, SQL embeddings) — highlighted
5. *Event-loop programming* (E in one vat) — unhighlighted
6. **Message-passing concurrent programming** (Erlang, AKL) — highlighted
7. *Imperative programming* (Pascal, C) — unhighlighted
8. **Imperative search programming** (SNOBOL, Icon, Prolog) — highlighted

For box 6, the poster stacks "Multi-agent programming" and "Message-passing concurrent programming" in one frame; for the excerpt, keep just the second name (it's the one your text uses) or stack both as the original does — either is defensible, but pick the one your §2.2.3 prose names.

Which edges (7). Six solid, one dashed:

- 1 → 2, labelled + *closure*
- 1 → 7, labelled + *cell (state)* — this is the long top edge in the poster, and it's what makes the excerpt a single connected story
- 2 → 3, labelled + *unification (equality)*
- 3 → 4, labelled + *search*
- 2 → 5, **dashed**, labelled ... — the poster reaches event-loop programming through intermediates (+ *name*, + *port*); the dashed arrow honestly elides them without asserting a false direct derivation. Verify the exact intermediate chain against the poster PDF before finalizing, and mention the elision in the caption.
- 5 → 6, labelled + *thread*

- 7 → 8, labelled + *search*

Note the small symmetry the excerpt makes visible that the full poster buries: + *search* is the concept that produces **both** Prolog boxes, from a declarative parent on one side and an imperative parent on the other — which is a rather elegant one-figure summary of why Prolog was never a single-paradigm language, and worth a sentence in the running text.

Highlighting device. Light fill plus heavier border on the three Web Prolog boxes — carrying over your rounded-rectangle annotation convention from the current figure keeps visual continuity, but since you now control the whole drawing, fill is cleaner than enclosure.

Here is a mockup of the proposed excerpt so you can judge the layout before redrawing it in your own tool:

continued after Claude tool activity

The layout in the mockup preserves the poster's left-to-right ordering (the "expressiveness of state" axis: the logic column leftmost, message passing centre, the imperative family rightmost), so a reader who later consults the full taxonomy will find everything where the excerpt taught them to expect it.

Suggested caption:

latex

```
\caption{Web Prolog located in Van Roy's taxonomy of programming
paradigms (excerpt, adapted from \citealt{van2009programming}). Boxes
are paradigms; an arrow labelled $+\$, \emph{concept}$ derives one
paradigm from another by adding that concept. The three highlighted
paradigms are those supported by Web Prolog: relational \& logic
programming and imperative search programming are where Van Roy places
Prolog itself, and message-passing concurrent programming is where he
places Erlang. The dashed arrow elides intermediate paradigms; readers
are encouraged to consult the full taxonomy.}
```

Companion text adjustment. The sentence "We have enclosed the relevant square boxes in boxes with rounded corners" refers to annotations that no longer exist; replace the figure-pointing passage with:

latex

```
\noindent In the very nicely organised and well argued taxonomy of
programming paradigms by Peter Van Roy \citep{van2009programming}, it
is possible to pinpoint exactly where Web Prolog fits. Figure~\ref{fig:paradigms}
reproduces the relevant excerpt. Note, in passing, a symmetry the full
taxonomy states but does not emphasise: \emph{search} is the concept
that produces both of the paradigms in which Van Roy places Prolog --
added to a declarative parent on one side and to an imperative parent
on the other. Prolog was never a single-paradigm language, and the
taxonomy says so twice.
```

This dovetails with the corrected three-model sentence I proposed last round ("relational & logic programming... imperative *search* programming... message-passing concurrent programming"), so the figure, the pointing text, and the enumeration finally all say the same thing.

Two verification tasks before you redraw. First, the dashed edge: confirm against the poster PDF exactly which intermediates sit between the functional column and event-loop programming (I believe + *name* and + *port* via ADT functional programming, but the excerpt's honesty depends on the elision being accurate about *what* it elides — if the chain differs, only the caption's "intermediate paradigms" phrasing needs to hold, which it will). Second, the box names: my mockup abbreviates ("Functional" for "Functional programming") for width; in your redraw at Springer's text width you have room for the full names, and matching Van Roy's wording exactly strengthens the "adapted from" claim.

On permissions, the concrete path: since the excerpt is redrawn in your own style and layout, the standard position is that no permission is required for the factual relationships it expresses — but send Van Roy a two-line courtesy email anyway (he is at UCLouvain and has been generous with this diagram for years), and keep his reply in the folder Springer's permissions questionnaire will eventually ask you to fill. That converts the caption's current "(Image source and license to be confirmed.)" into a solved problem rather than a deferred one.

That's all three. The natural next step on my side, whenever you're ready, is the same treatment for Chapter 3 — and if you redraw this figure or fix the n4.org code in the Chapter 1 diagram, send the revised versions and I'll re-verify them against the text.

Attachment hidden in shared chat The shared page does not expose the uploaded source file or image.

2 Jul

Claude Message 13

Before writing up Chapter 3, let me actually look at the two statechart PDFs — they arrived as extracted text only, and the topology is exactly what needs verifying.

All read — chapter text in full, all ten figures inspected (both statechart PDFs rasterized so I could see the topology, not just the extracted labels), and print-resolution checked across the batch. The headline: Chapter 3 is the best-organized chapter so far — the actor → toplevel → node → profiles → scenarios arc is genuinely good pedagogy, and the profiles section contains some of the manuscript's strongest thinking. But it also contains the deepest cross-chapter contradiction I've found yet (point 7 below), several transcript bugs of the kind Chapter 2 had, and two figures that tell different stories from their prose. In order:

Structure

1. An orphaned paragraph dangles after the summary's conclusion. The chapter's final paragraph ("The motivation for profiles is not only security, but also coordination. FYPB emphasises...") sits *after* the "overall message of the chapter" closer, separated by a `\medskip`. It reads as a displaced fragment — and it's good material, arguing that profiles answer FYPB's tooling-and-portability concern. It belongs in §"What are profiles good for?", where it would slot naturally after the security paragraph. Move it there and let the summary end where it ends.

2. The summary partly summarizes Chapter 2. The final security paragraph — "Like JavaScript, Web Prolog is designed as a sandboxed language: it omits file I/O, socket programming..." — recaps Chapter 2's §Web Prolog and security, not anything this chapter developed. And "Security in the actor model rests primarily on name distribution" is asserted here as if established, when the chapter body only gestures at it (the profile footnote explicitly *defers* capability-security to Chapter 4). Either cut the paragraph to the one sentence that is genuinely Chapter 3 material (profiles as declarations of security posture) or reframe it as a forward pointer. Also in the summary's last sentence: "how actors **migrate** and communicate across node boundaries" — does Chapter 4 propose actor migration? Remote *spawning* is not migration, and the word will set expectations (Erlang readers know the difference acutely). If migration isn't in the design, this word must go.

3. `mortal.pl` appears before it exists. §"The shared Prolog database" opens its example with "The contents of the file `\texttt{mortal.pl}` must be valid Web Prolog source code" — but the file is only introduced in the *following* subsection ("Deploying a node," where the owner runs `prolog-node --src=mortal.pl`). The reader meets a definite reference to an unintroduced artifact. Either swap the two subsections (deployment first, then what the shared database it loads may contain — arguably the more natural order anyway) or open with "Suppose the owner of the node `\texttt{n7.org}` keeps its knowledge in a file `\texttt{mortal.pl}`...". Relatedly, the `/node_info` paragraph mentions "the discovery hub" — a concept with no prior introduction anywhere; give it a three-word gloss ("a directory service introduced in Chapter~4") or cut the mention.

4. §3.1 pre-states what §3.2.2 then develops. The nutshell's "Private beliefs and shared knowledge" block gives the *complete* shadowing semantics — whole-predicate override, hidden-not-merged, restored-on-removal — and then §3.2.2 is an entire worked subsection re-stating exactly those three facts. The nutshell should preview, not preempt. Proposed trim for the nutshell block:

latex

```
\medskip
\noindent\textbf{Private beliefs and shared knowledge.}
Each agent may combine a private workspace -- including an optional
private dynamic database -- with shared, node-provided knowledge,
typically a read-only database maintained by the node owner. Private
definitions can \emph{shadow} shared ones, allowing local
specialisation without modifying the shared base; the precise
semantics of shadowing is worked out in
Section~\ref{sec:private-and-shared-knowledge}.
```

5. The PURE-profile material is duplicated. The "Conformance" subsection previews orthogonal profiles (PURE, statechart discipline) and promises "Section~\ref{sec:other-profiles} returns to this possibility" — but §"Could there be other profiles?" delivers nearly the same two examples with only the flat-SXML remark added. Either cut the preview to one sentence, or make the later section earn its existence by developing at least one orthogonal profile beyond name-dropping (a read-only/read-write axis, or a caching-guarantee profile, would each take a paragraph and genuinely extend the idea).

6. The signature-tables subsection is a stub with inventory gaps. Three tables, no prose, notation unexplained (the trailing `? in +Options?` presumably means optional — say so once), and coverage of only three of the six predicates. Meanwhile `respond/2` — used in the I/O example — appears in *no* inventory at all: not in the six-predicate list, not in the tables, not in the profile figure. And `output` is introduced as `/1` in the I/O subsection but invoked as `output/2` in the target-option section without comment. Either complete the tables (all six plus `input`, `output`, `respond`) or replace the subsection with one paragraph pointing to Appendix A, where this reference material more naturally lives.

Correctness and theoretical adequacy

7. The findnsols reconstruction contradicts Chapter 2's stated receive semantics — this is the chapter's most important finding. Chapter 2 (§semi-deterministic) states flatly: "The predicate `receive/1-2` is *semi-deterministic*, i.e. it either fails, or succeeds exactly once." But `wait_solutions/2` here works *only if that is false as stated*: its second clause body is a disjunction `( List = List0 ; toplevel_next(Pid), wait_solutions(...) )`, and the nondeterministic behaviour of the reconstructed `findnsols/4` — `L = [1,2,3,4,5] ; L = [6,7,8,9,10] ; ...` — requires backtracking *into the body of a completed receive* to take the second disjunct. If receive "succeeds exactly once" in the once/1 sense, that choice point is pruned and the reconstruction can only ever deliver the first slice. What you evidently mean, and what the implementation must do, is: *receive commits to its choice of message and clause — backtracking never re-selects — but choice points*

created by the clause body survive. That is a weaker and more interesting property than semi-determinism, and it needs to be stated precisely in Chapter 2, because this chapter's code (and Chapter 2's own claim that failure-driven loops are sometimes "the only way forward") depends on it. Proposed replacement for the Chapter 2 sentence:

latex

```
The predicate \texttt{receive/1-2} is \emph{committed} in its choice of
message and clause: once a message has been selected and a clause
fired, backtracking will never return to select a different message or
a different clause -- in that respect the operation is
semi-deterministic. Choice points created \emph{inside} the body of the
fired clause, however, survive as usual, and backtracking through them
is not only permitted but, as we shall see in
Chapter~\ref{sec:prolog-agents}, occasionally essential.
```

8. A pid bug inside a single continuous session. The `once` example is run against the same toplevel as every other example in the session (`Pid = 74122981`, spawned once and reused throughout), yet its flush shows `success(23091243, [101,102,103], false)`. Every neighbouring example correctly shows 74122981. Copy-paste from a different transcript; fix the pid.

9. The synchronous API reuses one options list for two incompatible option vocabularies. `toplevel_call_synch/4` passes the same `Options` to `toplevel_call/3` (which understands `template`, `limit`, `offset`, `once`, `target`) and to `wait_for_answer/3`, where it lands in `receive/2` (which understands `timeout`, `on_timeout`). The test then calls it with `[template(Who), limit(1)]` — so `receive` gets `template/limit` options. Either `receive` silently ignores unknown options — in which case say so, since ISO-minded readers expect option validation — or this errors. The clean fix is to split the argument (`CallOptions`, `WaitOptions`) or filter. Also worth one sentence: `wait_for_answer`'s guard `Answer if arg(1, Answer, Pid)` matches *any* message whose first argument is the pid — including `output` and `prompt` terms — so the "synchronous answer" may in fact be an intermediate event. For the exercises shown that never bites; for real use it's a bug, and flagging it would be honest.

10. `toplevel_halt/2` breaks the interaction model without comment, and swallows a down message. Every other predicate in the family is fire-and-forget with answers arriving as messages; `toplevel_halt($Pid, Reply)` binds `Reply = true` directly — a synchronous call implemented, presumably, by an internal send-plus-receive. One sentence should say so. Moreover, the toplevel was spawned with `monitor(true)`, so halting it must deliver a `down` message — yet no flush is shown and the `down` term never appears. Show it, or the attentive reader concludes monitoring silently lapsed. (Also consider a footnote on why the predicate is named *halt* while the protocol move is *exit* — presumably to avoid colliding with `exit/2` — since the mismatch is otherwise puzzling.)

11. "In traditional Prolog, a program cannot internally create a toplevel, pose queries and request solutions on demand" — too absolute, and your own citation refutes it. First-class *engines* do exactly this: Tarau's engine work — which you cite two pages later for the findall reconstruction — and SWI-Prolog's engines (`engine_create/3`, `engine_next/2`) let a program hold a suspended nondeterministic computation and pull solutions on demand. The honest claim is that *standard* Prolog offers no such facility, that engines exist as a system-specific answer, and that the toplevel actor generalizes the idea with a mailbox, a protocol, and network reach. That framing is *stronger* for your argument, not weaker: it positions the toplevel actor in a lineage rather than in a vacuum.

12. `assert/1` vs `assertz/1` — a standards problem for a standardisation book. ISO 13211-1 defines `asserta/1` and `assertz/1`; bare `assert/1` is a de-facto alias (SWI and others) but not ISO. This chapter (and Chapter 2, and the profile-hierarchy figure) consistently write `assert/1` while simultaneously invoking ISO authority ("Following the ISO standard, predicates that are modified this way need to be declared using the `dynamic/1` directive"). For a manuscript whose Chapter 12 argues for standardisation, this will be noticed. Decide once: either switch to `assertz/1` throughout, or declare `assert/1` an official Web Prolog alias in the language definition and say so at first use.

13. Goal-copying semantics is load-bearing and nowhere stated. The shadowing example quietly depends on it: the variable `Price` occurs both inside the spawned goal and in the parent's receive pattern, and the parent's binding arrives *only* via the message — because `spawn` copies its goal term into the child, severing variable identity. Erlang-style isolation requires exactly this, every example relies on it, and neither Chapter 2's `spawn` introduction nor this chapter says it. One sentence at `spawn`'s introduction ("the goal is *copied* into the new actor: bindings made by the child never flow back through shared variables, only through messages") closes the gap and pre-empts the single most common misconception a Prolog programmer will bring to `spawn/2`. Relatedly, the stray sentence "The precise form of the process identifier bound to `Self` is implementation dependent" after `Self = 34001234` seems vestigial — Chapter 2 already established the random-integer proposal — and I'd cut it.

14. Pids in GET query strings are capability tokens in URLs — flag it before a security reviewer does. The ISOTOPE design makes the pid the session capability and then transmits it as `GET /toplevel_call?pid=56120932&...`. Query strings are logged by servers, proxies, and browser history; leaking capability tokens via URL parameters is a textbook anti-pattern. Chapter 4 owns security, but this chapter *introduces* the mechanism, so it should carry one forward-looking sentence — e.g., that a deployment-grade mapping would move the pid to a header or POST body, and that the query-string form is used here for expository transparency. Absent that sentence, the design looks naive rather than simplified.

15. Two smaller semantic loose ends. (a) "Any other messages will be deferred to the code being executed by the toplevel" — "deferred to" is not a thing; the messages remain in the mailbox, available to receive calls made by the running goal itself. Rephrase. (b) In the PTCP walkthrough, `/down` is described as one of the messages the toplevel may send from `s2` (and again on `exit`), but `/down` appears in neither

statechart — reasonable, since it's the monitoring layer rather than the protocol, but say so in a caption or the prose ("monitoring messages are orthogonal to the protocol and not shown").

Terminology and consistency

16. Arity inconsistencies between text, figure, and examples. Text: `toplevel_stop/1`, `toplevel_abort/1`. Profile figure: `toplevel_stop/1-2`, `toplevel_abort/1-2`. Figure omits `toplevel_halt/2` entirely; text's six-predicate inventory omits `input/output/respond`. `input/2` in the prose vs `input/2-3` in the figure. Pick the authoritative arities (presumably the Appendix manual's) and reconcile all three surfaces.

17. The scenario pids don't reconcile where the text says they should. The browser-node subsection explicitly claims the browser-hosted session is transcript-identical to the remote ACTOR scenario ("we therefore do not repeat the traffic here") — and then cites pids that contradict the ACTOR figure: "the count server's `11719087`" vs the figure's `91883433`, and "`20071236@'http://n3.org'`" vs the figure's `65091236@'http://n3.org'`. Either the browser session is a *different* session (then drop "we do not repeat the traffic," which implies identity) or the pids must match. Also the robot scenario: prose JSON says `"pid": 47110023`, figure says `pid = 45012345`.

18. Assorted smaller items. "Cliopatria" here vs "ClioPatria" in Chapter 1 (official is ClioPatria; and consider "Wielemaker and colleagues" for credit). "Trinity demonstrator" (Ch1) — check this chapter's references stay consistent. "life-cycle" (section titles) vs "lifecycle" (caption, settings section) — pick one. "user-client" (once) vs Chapter 1's "end-user client." "bundle" appears exactly once, in Settings category 5, undefined — define it or write "loaded source files." `$label sec:actor-manager` for a section titled "The node controller" suggests a renamed concept; grep the book for stray "actor manager." Figure label `fig:browser-to-n1` for a scenario at *n9.org*. The Ch2 tension resurfaces: Chapter 2 defines `flush/0` in plain Web Prolog, this chapter says shell utilities like `flush/0` are "supplied by the node controller" — reconcile (e.g., "pre-loaded by the controller into every shell"). British/American: this chapter mixes worse than any so far — `initialise` and `initialize` appear in adjacent subsections, plus behavior (×6+), standardized/standardised both, authorized, specializes, organization. Grammar: "such the attempt" → "such as the attempt"; "The three phases... is illustrated" → "are"; "the client and the node starts" → "start"; "a HTTP header/response" → "an HTTP" (×2); "For example. if" → "For example, if"; "Examples such as these only exercises" → "exercise"; `"i.e \textbf{s2}"` → `"i.e." (×2)`. And the WebSocket lifecycle prose refers to phases "in blue" and "in black" — if the book prints greyscale, colour-dependent references break; use "first/second phase" with a pattern cue in the figure.

19. The HTTP/WebSocket primer paragraphs have Chapter 2's JavaScript problem. "Operating as a stateless request-response protocol, it supports methods such as GET... Evolving since the 1990s, HTTP has advanced through versions like HTTP/2 and HTTP/3..." — encyclopedic filler for an audience that knows HTTP. Compress each primer to the two or three sentences that carry design weight (statelessness and ubiquitous intermediary infrastructure for HTTP; bidirectional sub-protocols over a persistent connection for WS), and cut the JavaScript `onopen/onmessage` API tutorial. Happy to draft, same treatment as the JS passage.

The figures

20. PTCP statechart (core): topology verified correct, notation incomplete. The rasterized chart confirms exactly what the prose claims: `call s1→s2; /success(true) s2→s3; next s3→s2; stop s3→s1; /success(false), /failure, /error s2→s1`; and — the payoff of using a statechart at all — **abort** and **exit** drawn from the *superstate boundary*, correctly encoding "from any state." Three notational gaps: (a) no initial-state marker (dot-and-arrow into `s1`) — the same omission as Chapter 2's state-machine figure, and here it matters more because this chart claims to be a specification; (b) **exit** leaves the superstate into nothing — proper Harel/UML notation wants a final state (bullseye) or terminate pseudostate as its target; (c) the current caption ("client moves and toplevel responses") dropped the bold-vs-slash legend that your commented-out longer caption had — restore a minimal legend, since figures are read standalone: "Bold labels are messages and signals from client to toplevel; labels with a leading / are responses from toplevel to client." Two housekeeping items: both PDFs are true vector (good — this is what the PNG figures should also become) but carry Google Slides metadata titled "Kopia av Copy of PLAP_statechart3"; clean the metadata, and note "PLAP" is a fossil of an earlier protocol name — worth grepping the whole book and the demonstrator for survivors.

21. PTCP-over-HTTP statechart: correct, but the prose mischaracterizes `s5`. The chart adds `/output s2→s4` with `poll s4→s2`, and `/prompt s2→s5` with `respond s5→s2`, **abort/exit** still from the boundary — all consistent. But the text says "The additional states **`s4` and `s5`** ... are implementation states used to emulate push-style output **through repeated polling**," and then, two paragraphs later, correctly explains that prompts involve *no polling*. Only `s4` is the polling-emulation state; `s5` is an ordinary request-response exchange. Fix the sentence: "The additional state `\textbf{s4}` is an implementation state used to emulate push-style output through repeated polling requests; `\textbf{s5}` handles prompts, which follow an ordinary request--response rhythm. Both are part of the HTTP transport mapping rather than of PTCP itself."

22. life-cycle-kombi: caption promises a phase the figure lacks, and one arrow is unlabelled. The caption says "spawn, interact, terminate, and clean up" — there is no cleanup element anywhere in the drawing (and none in the prose either; if cleanup means the link-cascade reclamation of Chapter 2, *that* would be worth one visual element or a caption clause). Second: external **Send** and **Exit** are labelled yellow operations pointing *into* the loop, but the actor's own outbound message is an unlabelled arrow with an envelope — label it **Send** for symmetry, since the asymmetry implies (wrongly) that only inbound sending is an operation. Third, the text accompanying this figure repeats the Chapter 2 misframing that "the monitor option instructed *the actor* to send a down message" — same fix as before: the runtime notifies. Minor: how "New state" feeds back into "State" is carried only by the decorative circular arrow; a thin explicit return arrow would remove the ambiguity.

23. a-node: sound, one convention nit. Matches the prose component list exactly (controller, shared database, actors container, settings, web APIs), and extends Chapter 1's two-container node vocabulary consistently — good. The nit: the browser URL bar uses the placeholder `http://<domain>/shell` while the node's title bar says "HTTP domain name" — two different placeholder conventions in one figure; make the title bar `<domain>` too.

24. profile-hierarchy: the chapter's best figure, verified against the text almost perfectly. Ring contents match every textual claim I could test: stateless HTTP + owner predicates innermost ✓; ISOBASE adds ISO subset, `rpc/2-3`, `promise/yield` ✓; ISOTOPE adds `assert/retract/retractall`, `read/write/format`, `op/3`, semi-stateful HTTP ✓ (and this correctly predicts both the ISOBASE scenario's Unknown procedure: `assert/1` and the ISOTOPE scenario's successful assert — genuinely tight coupling); ACTOR adds concurrency + WebSocket ✓; and it confirms Chapter 1's "three of the four profiles operate entirely within the sequential model" and Chapter 2's "other profiles do not support `op/3`." Two fixes: the arity discrepancies of point 16, and one caption clause to defuse the one legitimate reader-stumble — `toplevel_*` predicates sit in the ACTOR ring, yet ISOTOPE is *defined* by *toplevel* actors: "Note that ISOTOPE exposes *toplevel* functionality through the semi-stateful HTTP API without exposing the `toplevel_*` predicates themselves, which become client-callable only at ACTOR."

25. The four browser scenarios: strong as a sequence, with one uncovered beat and one quiet design question. The RELATION figure's Unknown procedure: `=/2` is a beautifully chosen detail — the error message *is* the profile definition made visible (sharpen the prose to say so: "even unification is absent"; and fix "such the attempt"). The caption duplicates the question-marks sentence verbatim from the body — keep it in the caption only. The ISOBASE figure's `s --> [a], s.` timeout story checks out technically (no base case → no first solution → running timeout), and the browsable-source dashed box vs. RELATION's "?" is a nice visual rhyme; but the caption doesn't name ISOBASE while the neighbouring captions name their profiles — harmonize. The ISOTOPE figure contains a terminal beat the prose never covers: `?- read(T). _|:` goodbye. `T = goodbye.` — presumably `read/1` redefined via the prompt/respond machinery, which would be worth the two sentences it takes, since it completes the I/O story. And the same figure quietly poses a governance question the text should answer: the *shared* database declares `:- dynamic r/2.` while the client's assert lands in the *toplevel's private* database — is the owner's dynamic declaration what licenses client asserts of that predicate? If yes, that's an elegant policy mechanism deserving a paragraph; if no, the directive in the figure is superfluous and should go.

26. robot-to-actor-node: the figure and the text describe two different systems. The prose is emphatic that "the remote side does nothing clever — it simply echoes the input — so the interaction is a 'parrot' loop," spawning `echo_actor`. The figure shows a shared database labelled "% Social robot protocol," an actor with a "% memory" box in its private database, and that actor *spawning a child* — none of which an echo actor has or does. Even the speech bubble ("Hello, how can I help you?") is robot-initiated, which a parrot loop is not. Pick one story: either strip the figure to match the parrot (put `echo_actor`'s code in the dashed box, drop memory and the spawned child) or upgrade the prose to describe the richer agent the figure depicts — which, given that the section explicitly defers agent-level concerns to Chapter 8, argues for stripping the figure. Plus the pid mismatch of point 17, and the standing Furhat-photograph rights question from Chapter 1 applies here too.

27. Housekeeping across the batch. All PNGs are comfortably print-resolution this time (no failures; a-node and the scenario figures are generous). One figure is *referenced by no text at all*: `fig:a-toplevel-actor` is placed in §3.3 but never cited — either work it into the prose (it would sit naturally beside "A *toplevel* is a kind of actor, and what distinguishes it...") or cut it. And four referenced figures weren't uploaded: **prolog-agent-taxonomy-shells** (the one I most need to see — it must agree with Chapter 1's mindmap hierarchy shells → *toplevels* → actors, nodes → agents), `a-toplevel-actor`, `http-communication`, and `ws-communication`. Send the taxonomy at minimum.

What to protect

The offset/limit **stability contract** paragraph ("a client that cannot mutate cannot, by paging alone, violate the stability its own slices depend on") and the **two paging strategies** subsection are exemplary — precise, honest about the failure mode, and they cash Chapter 1's backtracking-as-pagination claim in full. The **once-as-resource-directive** analysis is the kind of design writing referees praise. "RELATION differs **in kind, not merely in degree**" and the conformant-node-without-a-Prolog-engine move is the interoperability thesis of the whole book in miniature. And the **browser-node** subsection — "the containment a remote session must arrange has, here, nothing left to arrange" — is the chapter's best prose. One opportunity in that neighbourhood: the penguins lineage is buried in an implementation aside, yet ISOTOPE is recognizably a rationalisation of the Penguins design you co-created; one explicit sentence claiming that lineage would strengthen both the credibility of the profile and the historical record.

Most valuable next steps, in order: fix the receive-semantics wording in Chapter 2 (point 7 — it's a specification-level fix, not a typo), reconcile the robot figure with its prose, repair the pid glitches, and move the orphaned FYPB paragraph. I can draft the compressed HTTP/WS primers, the reconciled robot-scenario prose, or the completed signature tables — and send the taxonomy figure when convenient and I'll close the loop on the agent hierarchy.

You Message 14

2 Jul

Re: I can draft the compressed HTTP/WS primers, the reconciled robot-scenario prose, or the completed signature tables

Do this please.

All three, in chapter order.

1. The compressed HTTP/WS primers

Drop-in replacements for the two subsections in §"Two transports — three web APIs". The parent section's application-layer footnote survives untouched.

latex

```
\subsection{The HTTP transport}\label{sec:http-transport}

The \textit{Hypertext Transfer Protocol} (HTTP) is the Web's stateless
request--response protocol: a client issues a request -- typically
\texttt{GET} to retrieve a resource or \texttt{POST} to submit data --
and the server answers, owing the client no memory between exchanges.
Figure-\ref{fig:http-communication} illustrates a client interacting
with a node in this fashion. Two properties of HTTP carry design
weight here. The first is statelessness itself, whose consequences for
the stateless API -- meaningful URIs, replayable requests, cacheable
responses -- are developed in
Sections-\ref{sec:browser-to-relation-node}
and-\ref{sec:statelessness-protocol}. The second is ubiquity: the
Web's vast infrastructure of proxies, caches, and other intermediaries
speaks HTTP, and its security machinery -- TLS encryption via HTTPS,
along with standard authentication schemes -- is inherited wholesale
by anything that rides on it. A Prolog node is, in this respect, just
another web server.
\end{verbatim}
```

latex

```
\subsection{The WebSocket transport}\label{sec:websocket-transport}

Where HTTP is request--response, WebSocket is a persistent,
bidirectional, low-latency channel: once a connection is established,
either party may send a message at any time, which is what makes
push-driven interaction possible. A connection begins life as an
ordinary HTTP request that is then upgraded to the WebSocket
protocol;\footnote{The protocol is specified in IETF RFC~6455; the
browser API is maintained by the WHATWG as part of the HTML standard.
For a general introduction we recommend \cite{lombardi2015websocket}.}
thereafter the parties converse according to a \emph{sub-protocol} --
an application-layer convention, riding on WebSocket, that fixes what
the messages mean. The stateful API offered by the most capable nodes
is exactly such a sub-protocol.
Figure-\ref{fig:ws-communication} shows the three phases of a
connection's life-cycle: the HTTP handshake and upgrade, the exchange
of sub-protocol messages, and the eventual close.

Subject to the value of an HTTP header or parameter, answers and other
messages arriving from the node are encoded either as JSON or as
Prolog text; clients written in languages other than Prolog would
normally request JSON. WebSocket client libraries exist for all common
programming languages, and every major browser exposes the API
natively to JavaScript -- which is all a web application needs in
order to hold a live conversation with a node.
```

(Delete the stray `\end{verbatim}` in the first block — artifact of my formatting.) Four notes. First, this removes the colour-dependent phase references ("in blue," "in black"); when you send the ws-communication figure I'll check whether it needs pattern cues for greyscale printing. Second, a catch made while compressing: the original prose promises "three phases" and then describes only two — the close phase was never mentioned; the draft names all three, but verify the figure actually depicts a close phase. Third, the footnote replaces the current W3C reference — w3.org/TR/websockets points to a *retired* specification; the protocol authority is RFC 6455 and the API authority is WHATWG, and citing the dead W3C TR in a book that elsewhere invokes W3C standardisation would be an easy referee hit. Fourth, the cut JavaScript handler tutorial (`onopen / onmessage / ...`) is genuinely gone rather than relocated; if you want it anywhere, it belongs in the appendix beside the API details.

2. The reconciled robot scenario

This draft commits to the direction I recommended: the *parrot story wins*, and the figure is stripped to match. The opening paragraph of the subsection ("In the previous scenarios, the shell and its toplevel actor served as the root of control...") stays untouched; the replacement runs from the figure sentence through the protocol exchange. The two closing paragraphs ("The WebSocket transport matters here..." and "In realistic conversational systems...") also stay, unchanged.

latex

```
Figure-\ref{fig:robot-to-actor-node} sketches a deliberately minimal
toy scenario in which a social robot interacts with a remote ACTOR
```

node over the stateful WebSocket API.

The robot itself provides speech recognition and speech synthesis: apart from a canned greeting produced locally when the connection is established, it does nothing but turn the user's spoken input into text, send that text to the node, receive a reply as text, and speak it back. In this toy setup the remote side does nothing clever either — it simply echoes the input — so the interaction is a ``parrot'' loop. The purpose is not to propose a useful robot skill, but to isolate the transport-level point: unlike the shell scenarios, the client is not a terminal driving a toplevel, and yet it can still engage in persistent, bidirectional interaction with a remote node.

At the protocol level, the scenario begins with the robot requesting that the node spawn an echo actor — an actor of exactly the kind defined in Chapter~\ref{sec:web-prolog}, whose source is resident in the node's shared database. The request is sent as a WebSocket message to the node controller (encoded here in JSON for the benefit of a non-Prolog client), and the node responds by returning the pid of the spawned actor:

```
\begin{verbatim}
{ "command": "spawn",
  "goal": "echo_actor",
  "options": "[monitor(true)]" }
\end{verbatim}
```

```
\begin{verbatim}
{ "type": "spawned",
  "pid": 47110023 }
\end{verbatim}
```

\noindent This pid becomes the capability by which the robot addresses the actor in all subsequent traffic. When the user speaks, the robot sends the recognised text to the actor, and the echo returns over the same connection:

```
\begin{verbatim}
{ "command": "send",
  "pid": 47110023,
  "message": "echo(self, 'Nice weather today')" }
\end{verbatim}
```

```
\begin{verbatim}
{ "type": "message",
  "data": "echo('Nice weather today')" }
\end{verbatim}
```

\noindent The atom \texttt{self} in the outgoing message is resolved by the node controller to an address standing for the connection itself, so that the actor's reply — sent, as always, to the pid it finds in the message — is routed back to the robot. The robot speaks the echoed text, and the loop repeats.

The important caveat: the `send` command and the `self`-resolution convention in the third and fourth JSON blocks are **my proposal, not your documented API** — the chapter as written promises that the pid "becomes the capability by which the robot can address the actor in subsequent messages" but never shows how a JSON client actually sends to a bare actor (all documented commands are `spawn` and the `toplevel_*` family). That gap is real: the ACTOR profile's direct-messaging path for non-Prolog clients needs *some* command vocabulary, and drafting the beat exposed it. Check the demonstrator — if it has such a command, substitute the real names; if it doesn't, this is a missing piece of the API surface, and the appendix should gain it. Either way the scenario needs the exchange shown, because the parrot loop is the entire point of the scenario and the current text never displays a single parrot exchange.

Figure edits to match (all subtractions, so quick in your drawing tool): replace the dashed "% Social robot protocol" box with the `echo_actor/0` source from Chapter 2; delete the "% memory" box inside the actor; delete the Spawn arrow and the child actor (an echo actor spawns nothing); relabel the pid to `47110023`; keep the mailbox/receive/process/send anatomy exactly as drawn, since it correctly reuses the Figure 1.4 vocabulary. The speech bubble may stay — the "canned greeting" clause in the prose now accounts for it. The Furhat photograph rights question stands from the Chapter 1 audit.

3. The completed signature section

Full replacement for §"Signatures and options for toplevel_ * predicates". It adds the missing notation key, covers all nine predicates instead of three, and appends the answer-term/JSON correspondence table that the scattered examples currently make the reader assemble alone.

latex

```
\subsection{Signatures and options for the toplevel API}
\label{sec:toplevel-signatures}

This section summarises the client-facing API; Appendix~\ref{sec:manual}
gives the full specification. In signatures, \texttt{+} marks an input
argument, \texttt{-} an output argument, and a trailing \texttt{?}
marks an argument that may be omitted (accounting for the double
arities such as \texttt{toplevel_spawn/1-2}).
```

```

\subsubsection*{\texttt{toplevel\_spawn(-Pid, +Options?)}}

\begin{tabularx}{11cm}{|X|X|X|}
\hline
\textbf{Option} & \textbf{Values} & \textbf{Default} \\
\hline
\texttt{session} & \texttt{true}, \texttt{false} & \texttt{false} \\
\texttt{target} & pid & caller's pid \\
\texttt{node} & URI & the local node \\
\hline
\end{tabularx}

\medskip
\noindent In addition, \texttt{toplevel\_spawn/2} accepts every option
that \texttt{spawn/3} accepts -- \texttt{monitor}, \texttt{link}, and
the \texttt{load\_*} family (Section~\ref{sec:private-db}).

\subsubsection*{\texttt{toplevel\_call(+Pid, +Goal, +Options?)}}

\begin{tabularx}{11cm}{|X|X|X|}
\hline
\textbf{Option} & \textbf{Values} & \textbf{Default} \\
\hline
\texttt{template} & term & \texttt{Goal} \\
\texttt{offset} & integer  $\geq 0$  & \texttt{0} \\
\texttt{limit} & integer  $\geq 0$  & all \\
\texttt{once} & \texttt{true}, \texttt{false} & \texttt{false} \\
\texttt{target} & pid & from \texttt{toplevel\_spawn} \\
\hline
\end{tabularx}

\subsubsection*{\texttt{toplevel\_next(+Pid, +Options?)}}

\begin{tabularx}{11cm}{|X|X|X|}
\hline
\textbf{Option} & \textbf{Values} & \textbf{Default} \\
\hline
\texttt{limit} & integer  $\geq 0$  & from \texttt{toplevel\_call} \\
\texttt{target} & pid & from \texttt{toplevel\_call} \\
\hline
\end{tabularx}

\subsubsection*{The remaining predicates}

\begin{tabularx}{\textwidth}{|l|l|l|}
\hline
\textbf{Predicate} & \textbf{Effect} & \textbf{Response} \\
\hline
\texttt{toplevel\_stop(+Pid)} & & abandons remaining solutions; PTCP returns to \texttt{s1} & none \\
\texttt{toplevel\_abort(+Pid)} & & signal; aborts the running goal from any state & \texttt{ack} \\
\texttt{toplevel\_halt(+Pid, -Reply)} & & signal; terminates the toplevel (synchronous) & \texttt{Reply}, plus \texttt{down} if monitored \\
\texttt{output(+Term)} & & called \textit{by} the running goal; emits an event to the target & \texttt{term} \\
\texttt{input(+Prompt, -Term)} & & called \textit{by} the running goal; requests input and blocks & \texttt{prompt} \\
\texttt{respond(+Pid, +Term)} & & called by the client; answers a pending prompt & none \\
\hline
\end{tabularx}

\subsubsection*{Answer and event terms}

A toplevel communicates with its target through a small vocabulary of terms, each with a JSON counterpart used when the client is not a Prolog process:

\begin{tabularx}{\textwidth}{|l|l|l|}
\hline
\textbf{Term} & \textbf{JSON type} & \textbf{Sent when} \\
\hline
\texttt{success(Pid, Solutions, More)} & \texttt{success} & a slice of solutions was produced; \texttt{More} is \texttt{true} iff further solutions may exist \\
\texttt{failure(Pid)} & \texttt{failure} & the goal failed \\
\texttt{error(Pid, Error)} & \texttt{error} & the goal raised \texttt{Error} \\
\texttt{output(Pid, Term)} & \texttt{output} & the goal called \texttt{output/1} \\
\texttt{prompt(Pid, Prompt)} & \texttt{prompt} & the goal called \texttt{input/2} \\
\texttt{down(Ref, Pid, Reason)} & \texttt{down} & a monitored toplevel terminated \\
-- & \texttt{spawned} & controller-level: an actor was created (carries the pid) \\
-- & \texttt{abort} & controller-level: a goal was aborted \\
\hline
\end{tabularx}

\medskip
\noindent The last two rows have no answer-term counterpart because,

```

```
in the Prolog API, the corresponding information is delivered through
argument bindings (\texttt{toplevel_spawn/2} binds the pid) or a
plain \texttt{true} (\texttt{toplevel_abort/1}).
\end{verbatim}
```

(Again strip the trailing `\end{verbatim}`.) Decisions I took that need your ratification, since I was completing a specification rather than copy-editing: I standardised on `toplevel_stop/1` and `toplevel_abort/1` — the arities the running text uses — which means the **profile-hierarchy figure must change** from `/1-2` to `/1` for both, unless the appendix defines a second options argument, in which case the text changes instead; the figure should also gain `toplevel_halt/2` and `respond/2`, both currently absent from its ACTOR ring. I added the `node` option to `toplevel_spawn`'s table because the ACTOR scenario uses it, and folded the spawn-inherited options into a prose line rather than table rows. The `prompt` and `down` JSON types in the correspondence table are extrapolated — the chapter shows Prolog `prompt/down` terms and JSON `success/failure/error/output/spawned/abort`, but never a `prompt` or `down` as JSON — so verify those two type names against the demonstrator before this table becomes normative. Finally, `input/2` vs the figure's `input/2-3` and `output/1` vs `output/1-2`: the table documents the arities the chapter demonstrates; whichever extended arities the appendix defines (presumably a `target`-bearing variant of `output`) should be reconciled in one pass across text, figure, and this section.

One follow-on the third deliverable creates: with the answer-term table in place, the puzzling detail in the I/O example — `success(74122981, [output(hello)], false)` containing the goal itself as the "solution" — becomes explainable in one sentence ("since `\texttt{template}` defaults to the goal, the solution list here contains the goal instance itself"), and I'd add that sentence right after the example. And the standing items from the audit remain queued on your side: the `once`-example pid, the Chapter 2 receive-semantics rewording, and the taxonomy figure whenever you're ready to send it.

You Message 16

2 Jul

Attachment hidden in shared chat The shared page does not expose the uploaded source file or image.

Chapter 4.

Claude Message 17

Chapter 4 read in full, both figures audited, resolution checked (both comfortably print-safe at `\textwidth`). The headline: this is the strongest chapter yet in its analytical writing — the backtracking-as-pagination development and the security triad are publishable centerpieces — but it contains the most *self-contradicting examples* of any chapter so far: three of its own transcripts violate rules the chapter itself establishes, and one architectural claim contradicts Chapter 3 directly. In order:

Correctness and self-consistency — the major catches

1. The node-resident services die with the owner's shell — the chapter's own containment rule kills its own examples.

`$Containment` establishes: `link(true)` is the default; `link(false)` must be passed explicitly and is reserved for owners. But the count service is started with `spawn(count_actor(0), Pid)` and the pubsub service with `spawn(pubsub_actor([]), Pid)` — no `link(false)` in either. Under the chapter's own semantics, both "durable, shared, continuously available" services are linked to the owner's shell session and are torn down the moment that shell exits. The examples must read:

latex

```
?- spawn(count_actor(0), Pid, [link(false)]),
   register_service(counter, Pid).
```

— and this is also the natural place to *showcase* the owner privilege: one sentence noting that `link(false)` is exactly the option clients are denied and owners need makes the example teach the role split instead of quietly violating it.

2. The count transcript is wrong about what arrives in the mailbox. `count_actor/1` (per Chapters 2 and 3, and per Chapter 3's own terminal showing `Shell got count(1)`) replies with `From ! count(C)`. The receive pattern here is `{Count -> true}` — a bare variable matching the whole message — so the binding is `Count = count(1)`, not `Count = 1`. Fix either side:

latex

```
?- counter@'http://n1.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 1.
```

3. The promise example queries a predicate its own figure says doesn't exist. `promise('http://n1.org', mortal(Who), Ref)` yields `success([mortal(plato), mortal(aristotle)], false)` — but this chapter's two-hop figure defines `n1.org`'s *entire* database as `human(plato).human(aristotle).`; `mortal/1` lives at `n2.org`. Change the goal to `human(Who)` — the answer then becomes `success([human(plato), human(aristotle)], false)`, verbatim identical to the `n1` response in the two-hop walkthrough, which is elegant rather than merely correct.

4. `load_predicates([pong/1])` ships a predicate that doesn't exist. The ping-pong figure (and the protocol it implements) defines `pong/0` — `pong :- receive({...})`, no arguments. The injection example must say `load_predicates([pong/0])`.

5. Promise/yield is assigned to the wrong API surface, contradicting Chapter 3. Chapter 3 is explicit: `promise/yield` belong to **ISOBASE** and "can all be implemented on top of the stateless HTTP API" — and the profile figure places them inside the ISOBASE ring, which need not offer the semi-stateful API at all. Yet §Timing's middle bullet, "**Semi-stateful HTTP API** — split-phase waiting," presents `yield/3` as that surface's characteristic operation. The truth is that `promise/yield` is *client-side* split-phase over an ordinary stateless request — the server sees a plain `/call`; the asynchrony lives in the caller. Proposed rewrite of the middle bullet:

latex

```
\item[\textbf{Split-phase waiting -- semi-stateful HTTP and
promise/yield.}]
Work submission and result retrieval are separated. Over the
semi-stateful API, a client advances a session and retrieves pending
responses through successive short requests, with idle and running
limits governing reclamation. The \texttt{promise/3-4} and
\texttt{yield/2-3} pattern has the same shape from the caller's side
even though it rides on the \emph{stateless} API: the pending
computation is simply an in-flight request, and \texttt{yield/3} with
a \texttt{timeout/1} option bounds the wait for it. In both cases,
waiting is explicit application logic — polling intervals, retries,
and fallback actions.
\end{verbatim}
```

(strip the stray `\end{verbatim}`).

6. The rpc-over-HTTP implementation rests on an unstated and fragile protocol assumption: variable correspondence by printed name. The code serializes `Goal` and `Template` in *two separate* `format/3` calls (`"(~p)"`), sends them as *two separate* URL parameters, and the server necessarily parses them with *two separate* reads — under which the textual `_123` in the goal and the `_123` in the template are different variables unless the server deliberately reconciles them **by name**. That reconciliation works only if the client printed consistent names across the two writes, which in SWI is an address-stability accident, not a guarantee, and `~p` (which honours `portray`) is the wrong directive for round-tripping in any case — `~q` is. Two fixes, one local and one architectural: locally, serialize goal and template as *one* term (`format(atom(GT), "q", [Goal-Template])`), one parameter, one read, shared scope); architecturally, whatever the wire protocol actually specifies — name-based matching across the `goal=` and `template=` parameters, presumably, since Chapter 3's API has the same two-parameter shape — must be *stated*, because it is a conformance-relevant protocol rule that two implementations could currently get differently. This is the kind of gap an independent implementer hits in week one.

7. `wait_answer/2` violates the chapter's own timing principle and exposes an unspecified language rule. Three sub-points. (a) The reference implementation spawns with `monitor(false)` and receives with no timeout — so a node crash or partition mid-enumeration blocks the caller *forever*, in a chapter whose stated principle is "place timeouts at the point where a process would otherwise block." One sentence suffices: "a production implementation would monitor the toplevel and bound the receive; we omit both for clarity." (b) The `failure(Pid) -> !, fail` clause contains a cut inside a receive-clause body, and the book has never defined whether receive bodies are transparent or opaque to cut. Here the cut is redundant at best (`->` already commits within the disjunction; receive itself commits), ill-defined at worst — and the question "what does `!` cut to inside `receive`" is a genuine language-specification item that belongs in Chapter 2 or the appendix, since Erlang offers no precedent. Delete the `!` here regardless. (c) This is now the *third* piece of load-bearing code (after `findnsols` and the toplevel sketch) that backtracks into a receive body's disjunction — the Chapter 2 semi-determinism rewording from my last round has graduated from advisable to mandatory. Also the Chapter 3 options-reuse smell recurs: `Options` is passed to both `toplevel_spawn` and `toplevel_call`; one global sentence in the language definition ("unrecognised options are ignored") would retire this whole class of flag.

8. `yield/3`'s timeout behaviour is non-steadfast and the prose references an option the example doesn't use. "On timeout, `on_timeout/1` is called and `yield/3` succeeds without binding `Answer`" — but the example passes only `timeout(0.3)`; no `on_timeout` anywhere. Fix the sentence or the example. Deeper: "succeeds without binding `Answer`" makes `yield/3` non-steadfast — call it with `Answer` already bound and the timeout path has no coherent behaviour — and "success with an unbound output" has no logical reading. A cleaner contract (fail on timeout, or bind a distinguished `timeout(Ref)` term) deserves at least a design-note paragraph defending whichever choice stands. Smaller: the polling loop prints "Still waiting..." *after* every yield including the successful one (the transcript faithfully shows this); moving the `writeln` inside the `var(Answer)` branch makes both code and transcript read sanely.

9. Three spellings of one exception term. The global-goals code catches `timelimit_exceeded`; Chapter 3's settings section throws `timeout_exceeded`; SWI-Prolog's actual term is `time_limit_exceeded`. Pick one book-wide (and appendix-wide) — this is precisely the kind of term an implementer will grep for.

Structure and exposition

10. `name@URI` addressing is introduced silently, and the FIFO caveat that depends on it is cryptic. The count example is the first-ever use of `counter@'http://n1.org'` — node-qualified *registered names* as an addressing mode — with no introduction, even though pids-with-@ were introduced ceremonially. Then §Uniform messaging states ordering as "per sender-receiver FIFO **when sender and receiver are addressed without mixing addressing modes**" — a parenthetical carrying real semantics (send twice to the same actor, once by pid and once by name, and ordering is forfeit) that no reader can decode without the two-modes concept being explicit. Add two sentences where the count example introduces the form, and one sentence in the FIFO paragraph explaining *why* mixing breaks ordering (different resolution paths, potentially different queues). Chapter 2's unconditional per-sender-FIFO statement then needs a forward-referencing qualifier.

11. `register/2` in the examples vs `register_service/2` in the doctrine. §Naming argues, correctly and firmly, that published services need dedicated operations — then both service examples, three pages earlier, use plain `register/2`. Either move the naming/discovery distinction before the examples, or use `register_service/2` in them with a forward gloss ("a dedicated publication operation, motivated below"). The current order teaches the anti-pattern first. Relatedly, that subsection cluster states the "no longer a process, now a service / part of the public surface" point **three times** (after the counter, after the pubsub, and in the "With this distinction in hand" paragraph); once with full force beats thrice with diminishing returns — I'd cut the final recap paragraph entirely.

12. Dangling and backward references. (a) "Later, we return to this observation when discussing the idea of a *pure* Prolog Web" — the chapter never does; the PURE profile lives in Chapter 3 and the purity theme presumably resurfaces in Chapter 9. Point the reference at its real target or cut it. (b) §Promise-yield says "In Section~\ref{sec:backtracking-as-pagination} we **will** return to this idea" — but that section precedes it; the tense and direction are inverted ("as argued in Section~..."). (c) §4.1's preview promises "orphan prevention" among its contents, but orphan prevention is one forward-pointing paragraph; the real treatment is §Containment. Soften the preview or add the explicit deferral. (d) "This implementation is provided in the transparent proof-of-concept implementation" — doubled noun, and "transparent" reads as a stray adjective; presumably "in the Trinity demonstrator's source code."

13. Security section: three sharpenings that strengthen its own argument. (a) *Inexpressibility* is described as enabling static rejection of dangerous code — but static sandboxing of a language with `call/N` and `=../2` is undecidable in general, and SWISH's sandbox-escape history proves the practical point. The chapter's *stronger* claim is available and is actually its own thesis: the dangerous predicates **do not exist** in the client-facing fragment, so a call to them is a trivially-detected existence error at zero analysis cost — absence, not analysis, is the mechanism. Rewriting two sentences turns an over-claim into the cleanest version of the argument. (b) The Armstrong name-discipline principle is a near-quotation with no citation; anchor it to the 2003 thesis. (c) The reply-channels subsection is excellent — and it indicts every example in Chapters 2–4, all of which embed `Self` in requests (`count(Self)`, `echo(Self, hello)`). One candid sentence ("the examples in this book use the simple idiom within a single trust domain; across trust boundaries, prefer reply endpoints") converts an unacknowledged inconsistency into demonstrated self-awareness. Also: two security topics this book has now *raised* remain unaddressed here — the pid-in-URL capability leakage from Chapter 3's ISOTOPE API, and the new one this chapter creates: `load_uri` in a stateless GET is a textbook **SSRF vector** (the node fetches an attacker-chosen URI). Both belong in this section, even as one paragraph noting the mitigations (capabilities out of query strings; owner policy restricting fetchable origins, connecting to Chapter 3's bundle-loading setting). (d) Housekeeping: the intro announces *three* deployment concerns, §Deployment delivers *four* (host isolation added), and the figure's deployment box lists a fourth different set ("reverse proxy — network isolation"); harmonize the enumeration across all three surfaces. And the T-800 epigraph — self-aware fun, but it's the only fictional attribution among otherwise real epigraphs; your call, though Springer copy-editors may query it, and "T-800 Model 101" is trademarked IP even in jest.

14. Smaller items. Section title "The node option works for toplevels too!" — drop the exclamation mark for `register`. `whereis/2` is used here having only ever been *mentioned* (Chapter 3's naming list); verify it's actually introduced in Chapter 2 or the appendix. The `parallel/1` equivalence claim needs one more caveat: when *several* goals fail or throw, sequential conjunction deterministically reports the first, while `parallel/1` reports whichever loses the race — equivalence holds only up to the choice among abnormal outcomes; and say explicitly that remaining goals are aborted on early failure. The pubsub client's `writeln("Waiting for a message ...")` uses a double-quoted string as a printable — the book has never fixed Web Prolog's `double_quotes` convention, and under ISO defaults this prints a code list; either use atoms or declare the flag once in Chapter 2 (a standardisation-relevant decision, not a style nit). The `as/2` operator in the remote-modules code needs an `op/3` declaration to parse. "Svensson et al.~\cite{...}'s" — use `\citeauthor` or restructure the possessive. The Ciao active-modules footnote deserves a real citation (Cabeza & Hermenegildo), and the logical/immediate update view discussion would be well served by citing Lindholm & O'Keefe (1987), the locus classicus for the terminology. British/American: `unauthorized`, `serialized` (transport bullets) vs `serialisation`/`serialising` elsewhere, `realizes` (layering list) vs `realises` (summary), `behavior` (`parallel/1` paragraph) — same global pass.

The figures

15. ping-pong-scenario: right structure, three code-level flaws. The topology is correct and well-coupled to the text — pinger and toplevel on `n1`, ponger on `n2`, WS connections, and the terminal showing only the pinger's output, which is precisely the I/O-capability asymmetry the prose explains (and that explanation, note, finally answers the output-routing question I flagged in Chapter 2; add a back-reference there). The flaws: (a) a **smart quote** in the pong code — `format('Pong finished')` closes with a typographic `'`, a syntax error if literal; classic Keynote auto-substitution, worth checking every code-bearing figure in the book for; (b) all the `format/1` calls lack `~n`, yet the terminal shows one line per message — either the node controller's output wrapping guarantees line-per-message (then say so once) or add `~n` to match Chapter 2's listing, but figure and Chapter 2 code should be character-identical either way; (c) `ping(N, Pong_Pid)`'s second clause has

no `N > 0` guard, so the predicate is correct only by clause ordering and determinism — cheap to fix, and Erlang-literate readers will notice its absence since the Erlang original would pattern-match on 0 the same way but never backtrack. Also: the browser URL reads `http://n1.org/ide` — see point 17.

16. interaction-isobase: the best-verified figure in the book so far. I checked it three ways and it passes all of them: internally (`n2`'s clause order — `human(socrates)` before the `rpc` clause — exactly produces the terminal's solution order `socrates, plato, aristotle`); against the walkthrough (the "Answer (Prolog)" vs "Answer (JSON)" arrow labels match the text's `format=prolog` node-to-node hop and JSON browser hop precisely); and **across chapters** — `n1 = {plato, aristotle}`, `n2 = {socrates} ∪ rpc→n1` is exactly the world in which Chapter 1's `rpc('http://n2.org', human(Who))` yields `socrates; plato; aristotle`, which it does. That three-chapter coherence is worth protecting through all future edits; I'd even add a footnote here rewarding the attentive reader ("readers who recall the query against `n2.org` in Chapter 1 can now see where each of its three answers came from"). One walkthrough nit: the displayed request to `n1` carries no `template=` parameter while the `rpc` implementation shown a page earlier always sends one — align the walkthrough's GET line with the implementation (or note the default-template shortcut).

17. `/ide` vs `/shell` — a book-wide endpoint inconsistency. Both Chapter 4 figures (and, I can now confirm looking back, the browser windows in Chapter 1's Figure 1.3) use `http://<node>/ide`, while Chapter 3's endpoint table and all four Chapter 3 scenario figures use `/shell`. One name must win everywhere — table, prose, figures, and demonstrator.

What to protect

The **backtracking-as-pagination** suite is the chapter's crown: the offset-surface/cursor-semantics distinction, the ISO **logical update view** yielding snapshot-consistent pagination "essentially for free" — a genuinely novel bridge between the Prolog literature and web engineering — and the disciplined honesty of the eviction analysis ("precisely what separates a convenience from a contract") and the caching trade-offs. The **inexpressibility/invisibility/containment** triad is a memorable, teachable security frame, and **reply-channels-as-capabilities** is sophisticated material rare in language books. The **ether**-based uniform semantics gives the concurrency story a real theoretical anchor. And the sequential-on-concurrent layering delivers exactly what Chapter 1 promised, in the promised order.

Priority order for fixes: the five example bugs (points 1–4 plus the `pong/1` arity), the promise/yield surface contradiction (5), the wire-protocol variable-correspondence gap (6), and the Chapter 2 receive-semantics rewording that three chapters now lean on. I can draft the corrected node-resident-services subsection end to end (spawns, registration, retrofitted naming discussion), the sharpened inexpressibility paragraphs, or the SSRF/pid-leakage addition to §Security — say which, and send Chapter 5 and the taxonomy figure when ready.

You Message 18

2 Jul

Re: I can draft the corrected node-resident-services subsection end to end (spawns, registration, retrofitted naming discussion), the sharpened inexpressibility paragraphs, or the SSRF/pid-leakage addition to §Security

Do both, please.

Claude Message 19

Both, in chapter order. The first deliverable turned up a new design-level finding while I was drafting it — flagged in the notes, because it's bigger than the subsection it surfaced in.

1. The node-resident services subsection, end to end

Full replacement from the count-actor subsection through naming/discovery. The section intro paragraph ("In addition to predicates in the shared database, the owner of a node may install...") stays as is.

latex

```
\subsection{A node-resident count actor}

We begin with a deliberately small example. Assume that the owner of
the node \texttt{n1.org} starts a count actor, detaches it from the
owner's own session, and publishes it under a stable name:

\begin{verbatim}
?- spawn(count_actor(0), Pid, [
    link(false)
]),
   register_service(counter, Pid).
\end{verbatim}

\noindent Two details of this little ceremony repay attention,
because together they are what turns a process into a service. The
first is \texttt{link(false)}. Under the default lifetime discipline,
the new actor would be tied to the life of the owner's shell and torn
down when that session ends — exactly the right behaviour for
```

client-created helpers, and exactly the wrong one for a service meant to outlive any particular session. Detaching an actor from the hierarchy that created it is precisely the privilege that Section~\ref{sec:containment} reserves for node owners; here we see what the privilege is \emph{for}. The second is \texttt{register_service/2}, a publication operation distinct from ordinary \texttt{register/2}; the distinction is developed in Section~\ref{sec:naming-discovery} below.

A published name can be addressed from other nodes by qualifying it with the node's URI, just as pids are: where \texttt{12345@'http://n1.org'} denotes the process with pid \texttt{12345} at \texttt{n1.org}, the term \texttt{counter@'http://n1.org'} denotes whatever process is currently published under the name \texttt{counter} there. A client on some other node can therefore interact with the service without ever learning its pid:

```
\begin{verbatim}
?- self(Self).
Self = 51230945.
?- counter@'http://n1.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 1.
?- counter@'http://n1.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 3.
?-
\end{verbatim}
```

The example is intentionally simple, but it already shows an important shift in perspective. This actor is not a client-local helper process created for one temporary task; it is a node-resident process with a stable public name, and it therefore behaves like a shared stateful resource. The fact that the second reply is \texttt{3} rather than \texttt{2} is not an accident of presentation, but evidence that the actor's state is shared across interactions: another client has incremented the counter in the meantime.

A node-resident publish-subscribe service

The same service perspective becomes even clearer in a publish-subscribe example. Consider the following actor:

```
\begin{verbatim}
pubsub_actor(Subscribers0) :-
  receive({
    publish(Msg) ->
      forall(member(Pid, Subscribers0), Pid ! msg(Msg)),
      pubsub_actor(Subscribers0);
    subscribe(Pid) ->
      pubsub_actor([Pid|Subscribers0]);
    unsubscribe(Pid) ->
      ( select(Pid, Subscribers0, Subscribers)
        -> pubsub_actor(Subscribers)
        ; pubsub_actor(Subscribers0)
      ).
  }).
\end{verbatim}
```

This actor maintains a list of subscribers. A \texttt{subscribe(Pid)} message adds a subscriber, an \texttt{unsubscribe(Pid)} message removes one, and a \texttt{publish(Msg)} message broadcasts \texttt{Msg} to all currently subscribed processes. On its own, this is just another actor definition. It becomes a \emph{service} when the owner — this time of the node \texttt{n8.org} — starts it as a durable, detached, node-resident process and publishes it:

```
\begin{verbatim}
?- spawn(pubsub_actor([]), Pid, [
    link(false)
  ]),
   register_service(pubsub_service, Pid).
\end{verbatim}
```

A client can then subscribe to the service and wait for messages from it:

```
\begin{verbatim}
?- self(Self),
   pubsub_service@'http://n8.org' ! subscribe(Self),
   repeat,
   writeln('Waiting for a message ...'),
   receive({
     msg(Message) ->
       format("Received: ~p~n", [Message]),
       fail
   }).
Waiting for a message ...
Received: hello
Waiting for a message ...
\end{verbatim}
```

Note the quiet work that network transparency is doing here: the subscriber list held by the actor at `\texttt{n8.org}` contains `\emph{remote}` pids, and `\texttt{Pid ! msg(Msg)}` delivers across node boundaries with the same syntax as local delivery. The message `\texttt{hello}` was received because some client published it to the same named actor:

```
\begin{verbatim}
?- pubsub_service:'http://n8.org' ! publish(hello).
true.
?-
\end{verbatim}
```

The important point is not the specific protocol, but the change in status. Once the actor is started by the node owner, registered under a stable name, and intended for repeated use by multiple independent clients, it should be understood as part of the node's public surface: no longer merely a process that happens to be running, but a shared conversational resource offered by the node.

This example also illustrates a useful contrast with the counter. The counter service exposes shared `\emph{state}`: what one client observes depends on what other clients have already done. The publish-subscribe service instead exposes shared `\emph{coordination}`: clients interact indirectly through a common mediating process. In both cases the essential pattern is the same -- a node-resident actor with a stable public name behaves as a service.

The implementation shown here is intentionally minimal. In practice, one would monitor subscribers so that dead ones are pruned from the list, place the service itself under supervision, possibly persist subscriptions in a more durable store, and perhaps attach access control or topic structure. Those are engineering refinements. The conceptual point is already visible in the small example: named node-resident actors provide a natural way to realise stateful services on the Prolog Web.

`\subsubsection{Naming, discovery, and service publication}`
`\label{sec:naming-discovery}`

The preceding examples published their services with `\texttt{register\_service/2}` rather than the ordinary `\texttt{register/2}`, and the distinction deserves comment. Ordinary registration is a convenience for private or client-local actors: a client may use `\texttt{register/2}`, `\texttt{whereis/2}`, and `\texttt{unregister/1}` freely for actors that it has itself created and named. Service publication is a different act, exercising different authority: the node owner places a name into the node's public service namespace, using dedicated operations -- `\texttt{register\_service/2}` and `\texttt{unregister\_service/1}` -- that clients do not hold. Clients may send to a published service name, but they do not thereby gain the ability to discover the service's pid, to enumerate what else is running on the node, or to remove the published name. This raises the practical question: how do clients discover which services a node makes available?

The standard design principle is to separate a stable, location-independent identifier from the service's current location `\cite{vanSteenTanenbaum2023DistributedSystems}`. The Prolog Web inherits this directly: a service name should not bind clients to a particular pid, host, transport endpoint, or deployment topology. Services may be restarted, moved, replicated, or versioned; the published name remains stable.

The safe pattern is to treat discovery as an owner-curated `\emph{publication mechanism}`: the node exposes a directory that lists only those services intended as part of its public surface. Discovery should resolve `\emph{published}` names, not reveal the existence of everything that happens to be running.

This is a design invariant, not merely a preference. If clients could enumerate all running processes, or if ordinary client naming operations exposed or removed published services, discovery would collapse into introspection and name discipline would be lost. The security implications of this principle are developed in `Section~\ref{sec:invisibility}`.

In the Trinity, the cleanest way to expose such a directory is to make it queryable through `\texttt{rpc/2-3}`. Concretely, a node that wishes to advertise services may provide a relation such as `\texttt{service/2}` in its shared database:

```
\begin{verbatim}
service(counter, meta(actor, protocol(count_v1))).
service(pubsub_service, meta(actor, protocol(pubsub_v1))).
\end{verbatim}
```

Richer metadata -- versioning, authorisation requirements, documentation URIs -- is a matter of extending the second argument, not of changing the mechanism. A client discovers advertised services with an ordinary goal-oriented query:

```
\begin{verbatim}
?- rpc('http://n1.org', service(Name, Meta)).
```

```

Name = counter,
Meta = meta(actor, protocol(count_v1)) ;
Name = pubsub_service,
Meta = meta(actor, protocol(pubsub_v1)).
?-
\end{verbatim}

\noindent This keeps the design uniform. Service discovery remains
declarative and compositional: clients can ask for all services,
filter by protocol, or combine discovery with other constraints in
the same query. And it does not weaken the security model, because
the node remains free to decide what it advertises, to require
authorisation for the directory predicate, and to ensure that the
directory reveals only what it has chosen to publish.

```

What changed and why. Both spawns gain `link(false)` with prose that converts the fix into pedagogy (the owner privilege shown at work); the count transcript's receive pattern now matches the `count(C)` protocol; `register_service/2` is used from the first example with a forward gloss, and the naming subsection is rewritten to *explain* what the examples did rather than legislate retroactively; `name@URI` addressing gets its explicit introduction; the pubsub owner node is named; `writeln` takes an atom, not a double-quoted string; the dead-subscriber refinement is added; and the third, redundant statement of the service-status point (the old "With this distinction in hand..." paragraph) is cut, along with the "(or service/3)" waffle. One pleasant side effect worth noticing: the count client's transcript now demonstrates the invisibility invariant *visibly* — the client interacts with the service from first message to last without a pid ever appearing.

A new finding this draft surfaced — bare pids cannot survive marshalling as designed. The transcript sends `count($Self)` where `Self = 51230945`, a bare integer, across a node boundary; the service then executes `From ! count(C)` with `From = 51230945` — a pid that means nothing at n1.org, where the client does not live. For the reply to route back, the pid must be node-qualified when it crosses the boundary — but the runtime *cannot* qualify it automatically, because a bare integer inside an arbitrary message term is indistinguishable from the number 51230945. Erlang avoids this precisely by making pids a distinct data type. This suggests a design amendment worth serious consideration: make the Web Prolog pid an **opaque term type** that merely *prints* like an integer. That solves the marshalling problem (the serializer can recognize and qualify pids on egress), and it also repairs a soft spot in the capability story — with pids as plain integers, `X is 51230944 + 1, X ! spy constructs` a capability by arithmetic, so pids are unguessable but not, strictly, unforgeable; as an opaque type they become both. Check what the demonstrator actually does here (my guess: it either qualifies by connection-level bookkeeping or the transcript has never been run cross-node); either way this belongs in Chapter 2's pid introduction and the appendix, not just here. Also for the inventory: `register_service/2` and `unregister_service/1` are now used, not merely proposed — the appendix manual and the ACTOR ring of the profile figure must list them.

2. The security section revisions

2a. The sharpened inexpressibility. Replace the second paragraph of §Inexpressibility (from "This is the price..." through "...at execution time.") with:

latex

```

This is the price of allowing strangers to run code on your machine:
the code must be incapable of causing certain classes of trouble. The
crucial word is \emph{incapable}, and it is worth being precise about
the mechanism. Inexpressibility is not achieved by analysing client
programs for dangerous behaviour — an approach that Prolog's
meta-call facilities (\texttt{call/N}, \texttt{=../2},
\texttt{atom\_to\_term/3}) render undecidable in general, and that
analysis-based sandboxes have historically struggled to make
watertight. It is achieved by \emph{absence}: in the client-facing
fragment, the dangerous predicates do not exist. No amount of term
construction can conjure a procedure that the fragment does not
define; a goal that names one fails with an ordinary existence
error, detected trivially and at no analysis cost — exactly as the
ISOBASE scenario of Chapter~\ref{sec:prolog-agents} showed for
\texttt{assert/1}. A node may, as a courtesy, additionally scan
injected source at load time and reject code that references
undefined procedures outright, but such a scan is a diagnostic
convenience, not the security boundary. The boundary is the fragment
itself.

```

The first and third paragraphs of the subsection stand unchanged; the third ("Even when an operation exists in the underlying system, it is exposed... through owner-controlled predicates...") now sets up the SSRF material perfectly, so append immediately after it:

2b. The SSRF subsection, closing §Inexpressibility:

latex

```

\subsection{The limits of absence: mediated capabilities and
request forgery}

Absence handles the direct threat, but not the indirect one. Every
owner-mediated operation is a place where client intent is carried
out with the \emph{node's} authority, and the canonical web instance
is the \texttt{load\_uri} option. A request such as

```

```
\begin{verbatim}
GET /call?goal=solve(X)&load_uri=http://n5.org/planner
\end{verbatim}
```

\noindent instructs the node to fetch a client-chosen URI. From the network's point of view, that fetch originates from the node: it is issued from the node's network position and, potentially, with the node's credentials. A hostile client can exploit this to make the node probe or retrieve resources the client could not reach directly -- intranet hosts behind the node's firewall, cloud metadata endpoints, services that trust the node's address. In web-security terms, an unconstrained `\texttt{load\_uri}` is a `\emph{server-side request forgery}` (SSRF) vector, and the node an unwitting confused deputy.

The remedy is policy on the mediated capability, not a change to the language. A node should treat outbound fetching as configurable authority: restricting `\texttt{load\_uri}` to an owner-maintained allowlist of origins, refusing URIs that resolve to private address ranges or to the node's own metadata services, and disabling the option entirely for unauthenticated clients. The bundle-loading policy of Section~\ref{sec:node-settings} is the natural home for such rules -- a node that requires loaded bundles to be self-contained, or to come only from designated origins, has already closed most of this surface. The lesson generalises: whenever a mediated capability turns client input into node-originated action, the mediation must include an authorisation decision about the `\emph{target}` of the action, not only about the identity of the client.

2c. The capability-leakage subsection, placed inside §Invisibility immediately after "Reply channels as capabilities":

latex

```
\subsection{Capabilities on the wire}
```

Name discipline also has a deployment-level failure mode that deserves explicit mention. A pid is a bearer capability: whoever holds it can exercise it. Unguessability protects the capability against `\emph{search}`, but not against `\emph{disclosure}` -- and ordinary web infrastructure discloses URLs promiscuously. The semi-stateful HTTP API of Chapter~\ref{sec:prolog-agents} carries the session pid as a query parameter:

```
\begin{verbatim}
GET /toplevel_call?pid=56120932&goal=q(X)
\end{verbatim}
```

\noindent Query strings of this form are routinely written into server access logs, proxy logs, and browser histories, and may leak onward through Referer headers. Every such record is a stored copy of a live capability; the unguessable-pid argument is void for a token that the deployment itself writes to disk.

The exposition in this book uses the query-parameter form because it makes the protocol transparent. A deployment-grade mapping should move session capabilities out of the URI -- into a request header or a POST body -- so that they never enter the logging path. Two properties already present in the design limit the damage of any leak that does occur: pids are ephemeral, bounded by the idle and running timeouts of Section~\ref{sec:node-settings}, so a capability recovered from a log is likely dead; and possession of a pid is necessary but not always sufficient, since a node may require authorisation for operations performed through it. Neither property, however, is a reason to log capabilities in the first place.

2d. One bonus sentence for the end of the reply-channels subsection -- the candid acknowledgment from my critique's point 13c, which costs three lines and buys demonstrated self-awareness:

latex

It should be acknowledged that the running examples of this book -- `\texttt{count(Self)}`, `\texttt{echo(Self, hello)}`, and their kin -- use the simple idiom throughout, embedding the sender's own pid in the request. Within a single trust domain this is the natural convention, and we retain it for clarity; across trust boundaries, the discipline described here should take its place.

Placement rationale and residuals. SSRF lands in Inexpressibility because it is that section's own logic completed: absence closes the direct channel, so the residual risk migrates to the mediated ones, and `load\_uri` is the mediated capability par excellence. Pid leakage lands in Invisibility because it is name discipline's failure mode when names meet HTTP logging. The triad stays a triad -- both additions present themselves as the limits and deployment conditions of an existing mechanism, not as fourth and fifth mechanisms. Three residuals remain on your side: the deployment-concerns count mismatch (intro says three, section delivers four, figure lists a different four) is untouched by these

drafts and still needs its one-pass harmonization; "analysis-based sandboxes have historically struggled" is deliberately unnamed in my draft, but a footnote citing the SWISH/ `library(sandbox)` experience (Wielemaker's SWISH papers discuss it) would give the claim teeth — your call, since it names a friend's system as the cautionary example; and if you adopt the pid-as-opaque-type amendment from deliverable 1, the Invisibility subsection's "pids are chosen from a sufficiently large, unpredictable space" sentence should be upgraded to say the type is opaque as well as the value unguessable — at which point unforgeability becomes literally true rather than approximately true.

You Message 20

2 Jul

Attachment hidden in shared chat The shared page does not expose the uploaded source file or image.

Chapter 5.

Claude Message 21

Chapter 5 is code-dense enough that reasoning alone won't settle several questions — the toplevel implementation is built on specific SWI-Prolog behaviours (`findnsols/4` chunk boundaries, `count/1` limits, `nb_setarg/3` interactions with backtracking) that I can test empirically. Let me install SWI-Prolog and run the chapter's actual code before writing the critique.

continued after Claude tool activity

Empirical verdict is in — and it cuts both ways. Full critique, leading with what the machine says.

What I ran and what it found

I installed SWI-Prolog 9.0.4 and executed the chapter's `slice/5`, `answer/5`, `add_pid/3`, and a faithful simulation of the `state_1/state_3` control flow. (One methodological confession that is itself a finding: my first simulation used arity-2 answer terms against the book's `arg(3, Answer, true)` test, which *silently failed* and produced phantom bugs — the `arg/3` idiom's silence on arity surprises trapped its first hostile reader within the hour. More below.)

Verified correct — the book's transcripts are real. The `count(2) → nb_setarg → count(5)` transcript reproduces *character-for-character*: `success([1,2],true), success([3,4,5,6,7],true), success([8,9,10,11,12],false)`. And the case I most suspected — solution count an exact multiple of the chunk size, where a naive `findnsols` would leave a doomed choicepoint and a client's ``next'` would collapse the toplevel — is handled: 4 solutions in chunks of 2 yields ``success(4711,[1,2],true)` then `success(4711,[3,4],false)`. SWI's `findnsols` + `call_cleanup` detects exhaustion at the boundary. Two consequences for the text: this is an *implementation property* of SWI, not an ISO guarantee, so (a) one sentence should state that the protocol's liveness leans on it, and (b) it belongs in the regression suite, since a future `findnsols` that leaves a boundary choicepoint would make `answer/5` **fail** on the final `next' would collapse the toplevel — is handled: 4 solutions in chunks of 2 yields `success(4711,[1,2],true)` then `success(4711,[3,4],false)`. SWI's `findnsols` + `call_cleanup` detects exhaustion at the boundary. Two consequences for the text: this is an *implementation property* of SWI, not an ISO guarantee, so (a) one sentence and kill the session actor with no message to the client.`

Bug confirmed — `state_3`'s target redirection is broken as printed. In `state_1`, `Target = target(Target1)` is constructed *after* `state_2`, i.e., after the choicepoint that `$next`'s `fail` backtracks into. The binding is therefore unwound and rebuilt fresh from the original `Target1` on every redo, and the `nb_setarg(1, Target, NewTarget)` performed in `state_3` is discarded. Empirically, book shape: chunks 2 and 3 sent to `shell, shell`, despite redirection; fixed shape — moving one line, `Target = target(Target1)`, to just after `Limit = count(Limit0)` and before `state_2` — chunks 2 and 3 go to `redirected`. This matters normatively: Chapter 3's `toplevel_next` options table documents `target` as supported, so the reference implementation silently drops a documented feature. The fix is a one-line move; note in passing *why* `Limit` works and `Target` doesn't (Limit's `count/1` cell is built before the choicepoint and survives redo; that asymmetry is the entire trick, and stating it explicitly would teach the technique properly).

Bonus hazard found en route. My first test built `target(shell)` as a ground literal, and `nb_setarg` mutated the *compiled clause* — a documented SWI danger the chapter's technique actively invites imitators into. One warning sentence where `nb_setarg` is introduced: never apply it to a term that occurs as a source-code literal.

Correctness and semantics beyond the tests

1. The down-reason vocabulary now contradicts Chapter 2 directly. `par_yield` and `first_solution` pattern-match `down(_, _, exception(E))`, and the `sleep(a)` transcript (which rethrows `error(type_error(float,a), ...)`) proves the implemented convention *wraps* uncaught exceptions in `exception/1`. But Chapter 2's transcript delivered a crash reason as a **bare** term: `down(..., existence_error(proc, q/1))`. Under Chapter 2's convention, this chapter's code hangs (the pattern never matches, the down is deferred forever); under this chapter's convention, Chapter 2's transcript is wrong. The wrapped form is also the better design — it disambiguates a genuine crash from a user's `exit(Pid, false)` or `exit(Pid, exception(x))` spoof — so I'd standardize on: `true` (normal), `false` (failure), `exception(E)` (uncaught

exception), any other term (via `exit/1-2`), fix Chapter 2's transcript to `exception(error(existence_error(...),_))`, and put the vocabulary in the appendix. Note also that the `sore(milk)` transcript *confirms* failure-reason `false`, closing the documentation gap I flagged in Chapter 2 — reason `false` is now demonstrated twice and defined nowhere.

2. The timeout-expiry semantics of `receive/2` is undefined, and this chapter's cleanup code is hostage to it. `drain_mailbox/1` recurses until the mailbox has no matching messages, terminating only via `timeout(0)` expiry. If expiry makes `receive/2` fail, then `drain_mailbox` fails, so `tidy_up_all` fails, so in `par_yield`'s exception branch `tidy_up_all(Pids)`, `throw(E)` never reaches the throw — `parallel/1` fails where requirement 4 demands it *rethrow*. If expiry makes `receive/2` succeed doing nothing, everything works — but then Chapter 4's `yield/3` succeed-with-unbound-Answer behaviour follows for every timed receive, including `server_yield`, with the steadfastness problem I flagged there. Three chapters' code now leans on a semantics stated nowhere. Chapter 2 needs a `receive/2`-options paragraph that decides: on expiry, receive runs the `on_timeout` goal, whose default is `true/fail` — pick one — and `drain_mailbox` should then be written to be correct under the chosen rule (with `on_timeout(true)` explicit, if the default is fail).

3. Third receive-specification gap: clause-selection priority. `supervisor_loop`'s first two clauses overlap — `down(_, Pid, true)` and `down(_, Pid, Why)` both match a normal-termination message — and correctness depends on the *textually first matching clause* winning. Erlang guarantees this; Chapter 2's scan rule ("the first message that matches a pattern in any of the receive clauses") specifies message selection but not clause priority. One sentence in Chapter 2 closes it. (Running tally of receive-semantics items now owed to Chapter 2: committed-choice-with-body-choicepoints, guard-failure deferral, timeout expiry, clause priority, and cut transparency from Chapter 4.)

4. The supervisor's loop contradicts its own policy definitions. The text defines Erlang's actual triad — "*permanent* children are always restarted, *transient* only if abnormal, *temporary* never" — and then `supervisor_loop`'s first clause unconditionally *removes* any child that terminates normally. A permanent child that exits normally must be restarted under the stated definition; here it is silently dropped, policy never consulted (the abnormal path presumably consults it inside the unshown `restart_child`, but the normal path bypasses policy structurally). Either route normal downs through the policy too, or weaken the stated definitions to what the sketch implements — but don't state OTP's semantics and implement different ones, because OTP-literate readers check exactly this.

5. Hot code swapping's motivating example defeats its own mechanism. `'upgrade'(Pred1)'keeps'Stateo'unchanged—correctandclearlyexplained—` but the example motivates `'fridge2'as"perhapsusingabetterdatastructurerepresentingthestate,"whichispreciselytheupgradethat*cannot*keepthesta` keeps `'Stateo'` unchanged — correct and clearly explained — but the example motivates `'fridge2'` as "perhaps using a better data structure representing the state," which is precisely the upgrade that **cannot** keep the state unchanged. Erlang meets this with `'gen_server's'code_change/3'` state-migration callback. Either re-motivate ("*a more efficient algorithm over the same state representation*") or extend the message to `'upgrade'(Pred1)'keeps'Stateo'unchanged—correctandclearlyexplained—` but the example motivates `'fridge2'as"perhapsusingabetterdatastructurerepresentingthestate,"whichispreciselytheupgradethat*cannot*keepthesta` erver's'code change/3'state-migrationcallback. Either re-motivate ("*a more efficient algorithm over the same state representation*") or extend the message to `'upgrad` MigrationGoal` and spend a paragraph — the second option is more honest about why OTP's upgrade machinery has the shape it has, and one sentence citing `code_change` earns credibility either way.

6. `monitor/2` is introduced by two chapters pointing at each other. This chapter: "the `monitor/2` predicate introduced in Chapter 2." Chapter 2: `monitor/2` deferred to *this chapter*. Neither ever gives its signature or semantics. Introduce it properly here (`monitor(+Pid, -MonRef)`, `demonitor(+MonRef)`), fix Chapter 2's forward reference (and verify its arity — I recall Chapter 2 saying "`demonitor/2`"), and — the substantive part — **specify monitoring a dead pid**. The text honestly names the race ("a server could terminate in the small gap between obtaining its pid and installing the monitor") but leaves it open, when Erlang closed exactly this race decades ago: monitoring a dead process delivers an immediate `DOWN` with reason `noproc`. If Web Prolog adopts that (it should — otherwise the fail-fast abstraction can still block forever, and the section's whole point dissolves), the race paragraph turns from acknowledgment into resolution. Add `make_ref/1` to the same introduction — it's used here and inventoried nowhere — and note that `tidy_up`'s `demonitor(Pid)` works only via the spawn-option convention `MonRef = Pid`, worth one clause.

7. Transcript and demo defects in the fail-fast section. (a) `server_upgrade(Pid, fridge2)` — missing dollar: `$Pid`. (b) The error line `ERROR: Unhandled exception: Unknown message: server_down(false)` contains a stray "Unknown message:" — the shell would print `Unhandled exception: server_down(false)`. (c) The demo spawns with `monitor(true)` and the fail-fast machinery installs `monitor/2` — two monitors, two `down` messages, one consumed by `server_yield` (the throw), one surfacing in the subsequent `flush`. That's actually consistent, but unexplained it reads as a ghost; either drop `monitor(true)` from the spawn (cleaner demo) or add the sentence explaining the leftover. (d) The `Response0` indirection in `server_yield` is a missed teachable moment: receiving into a fresh variable prevents a caller-instantiated `Response` from *influencing message selection* — one sentence turns puzzling code into a lesson about patterns-with-bindings.

8. `thread_signal/2` is an unmarked abstraction break. `toplevel_abort/1` reaches below Web Prolog into SWI's thread API — treating a pid as a thread id and delivering an asynchronous exception. That's legitimate PoC engineering, but it's the *only* place in three chapters of implementation where the actor abstraction is pierced, and it deserves the sentence that says so: abort requires a host-level asynchronous signal; the actor primitives alone cannot express it. (This also quietly documents the PoC's actor $\leftrightarrow$ thread mapping, which readers of Appendix C will want.) Similarly `10 000 000 000` in `state_1` is not legal syntax — SWI wants `10_000_000_000` (itself a non-ISO extension worth a footnote) or plain digits, and Chapter 4's HTTP implementation used plain digits; harmonize the magic number's spelling, or better, name it once.

9. `tidy_up_all`'s drain has an inherent in-flight race — and Chapter 4 already owns the vocabulary for it. A worker killed by `exit/2` may have a message or its `down` in the ether at drain time; `timeout(0)` cannot wait for it, so it lands after `parallel/1` returns — precisely the pollution drain exists to prevent. Erlang's answer is `demonitor(Ref, [flush])`. One paragraph acknowledging the race in ether terms (a lovely payoff for §4.1's two-step model) and either adopting flush-semantics for `demonitor` or a small bounded timeout would make the cleanup story honest rather than apparently airtight.

10. The `concurrent/3` comparison is apples to oranges, and the orange's author is a likely reviewer. `parallel/1` spawns one actor per goal; Wielemaker's `concurrent/3` schedules M goals over N pooled threads — a strictly harder problem. "Simpler and more comprehensible than Wielemaker's" is true partly because it does less, and the sentence as written both overreaches and needles a colleague whose library the PoC elsewhere depends on. Reframe: comparable *per-goal* semantics, genuinely less scaffolding thanks to unification-based correlation — "though it must be said that `concurrent/3` additionally solves worker pooling, which `parallel/1` does not attempt."

Structure

11. The chapter fully delivers Chapter 1's promises — a first. Servers, supervisors, toplevels, `parallel/1`, `first_solution/2`: every item in Part II's advertised manifest is here. Protect that. One vestige to fix in Chapter 4: "introduced in the next chapter and implemented in Chapter~\ref{sec:implementing-prolog-generics}" — both references resolve to this same chapter; presumably a fossil of a draft where introduction and implementation were split. Also fix my earlier Chapter 2 roadmap replacement's assumption: confirmed correct, the hot-swap server and supervision did land here.

12. The `findnsols` presentation duplicates Chapter 3 verbatim — and misses the chapter's best available insight. Same borrowed SWI-manual example, same output, same footnote URL as §3's reconstruction section. But look at what the two passages jointly say: Chapter 3 built `findnsols/4` out of toplevel actors; this chapter builds toplevel actors out of `findnsols/4`. The constructions are mutually inverse — each abstraction is definable from the other — which is a genuinely elegant equivalence the book currently states in neither direction. Replace the duplicated tutorial with a back-reference plus two sentences making the inversion explicit; it upgrades a redundancy into an argument.

13. Terminology: the chapter's central distinction leaks. The intro defines *Erlang-style behaviors* vs *Prolog-style generics*, then immediately calls `rpc/2-3` "a Prolog-style **behavior**" — and Chapter 4 calls `parallel/1` the same. Pick the discipline: *generic* is the umbrella, *behaviour* is reserved for the Erlang-style protocol-bearing kind, and the Prolog-style ones are generics or meta-predicates. And on spelling: this chapter oscillates between "behavior" (section titles, intro) and "behaviour" (running text, summary) — the strongest possible tiebreaker exists here, because Erlang itself spells the construct `-behaviour` in its module attribute; the book's British convention and Erlang fidelity point the same way. Same pass catches `specialized/specialised` coexisting in adjacent paragraphs.

14. Smaller structural items. The statechart figure is the *same PDF* already shown in Chapter 3, now recaptioned "The complete Prolog Toplevel Communication Protocol" — Springer discourages duplicated figures; the text already says "consider *again*," so a `\ref` back to Chapter 3's figure serves better (and the new label contains the typo "complate"). The implementation set silently omits `toplevel_halt/2`, `input`, `output`, and `respond` — add a scope sentence ("the remaining predicates involve the node controller and are covered in Appendix..."), especially since my Chapter 3 signature tables now document them. The supervisor footnote contains a literal `\url{https://...}` TODO placeholder — must not survive to submission — and "sixteen passing tests" is a count that will rot; "a test suite" ages better. The intro lists statechart actors among Erlang-style behaviours without noting they're Chapter 6's subject — add "(the subject of Chapter~\ref{...})". A large commented-out block of superseded prose sits before `parallel/2` — source hygiene before handing the manuscript over. The `fridge/4` gloss "a tiny key-value store where the keys are food items" mischaracterizes a multiset ("a tiny store whose state is a multiset of food items"). And verify the claim "in Chapter 2 we modelled the behaviour of a fridge *directly as a Prolog relation over states*" against Chapter 2's actual fridge, which I recall as a receive-loop actor with interleaved logic — the truthful contrast is "there, protocol handling and domain logic were interleaved in one loop; here the domain logic is factored into a relational callback," which is also the *stronger* contrast for your argument.

15. Credit the `count/1` mechanism to its source. The text presents the mutable-limit trick as "one feature of `answer/5`," but it is SWI's documented `findnsols/4` convention (`count(N) + nb_setarg`). Say so and cite the manual entry — otherwise implementers on other systems won't know the semantics has a home, and readers will credit the wrong author. More broadly, one framing sentence for the whole toplevel section: `nb_setarg`, `findnsols`, `thread_signal` are the SWI PoC's *techniques*; the PTCP is the *contract*; other implementations may realize §3's limit/target updates entirely differently.

16. Typos and grammar. "built-in" → built-in; "implemmentation of session/3 like so:." → implementation, single colon; "The most important part... are" → agreement; "if one of the goals fail" → fails; "that Wielemaker's version support" → supports; the `offset/2` footnote lacks `\url{}` wrapping (its sibling has it); "post creation-time" vs Chapter 1's "after creation-time."

What to protect

The `parallel/2` → `parallel/1` → fail-fast progression is the best pedagogy in the book so far, and it contains two quiet triumphs worth spotlighting rather than leaving implicit. First, the goal-copying statement ("the goal in the first argument of `spawn/1-3` is always copied before being called") — this is the *first explicit statement in the manuscript* of the semantics I flagged as missing in Chapter 4; it belongs in Chapter 2 at `spawn`'s introduction, with this passage back-referencing it. Second, `par_yield`'s fail-fast behaviour works because message-major selection picks the *oldest* message matching *any* clause — a crashed worker's `down` preempts a healthy worker's pending reply — which is Chapter 2's scan rule paying rent; one sentence pointing at it converts an invisible dependency into a demonstration. Likewise, the nested `state_3` receive gives you Chapter 3's documented message-deferral behaviour (a `$call` arriving in `s3` waits for `s1`) *for free* — say so, it's the implementation validating

the specification. And the summary's closing argument — `maplist` as orchestration, unification as correlation, goals-as-terms as work distribution, `fail` as cleanup trigger, "none of them designed for concurrency, yet they compose" — is the chapter's thesis stated exactly right, now with empirical backing: the one bug I found (`Target` placement) is itself a lesson in the composition's one sharp edge.

Priorities: the `state_1` one-line fix (verified), the down-reason vocabulary standardization (cross-chapter), the receive/2 semantics package for Chapter 2 (now five items), the supervisor policy contradiction, and the dead-pid monitor semantics. I can draft the consolidated Chapter 2 "Semantics of receive/1-2" subsection covering all five owed items, the corrected supervisor loop with policy-aware normal-termination handling, or the `monitor/2`/`demonitor/1`/`make_ref/1` introduction block — say which. And the taxonomy figure from Chapter 3 is still outstanding whenever convenient.